

Model Evaluation & Improvement

Dr. Benoit Mialet

benoit.mialet@hotmail.fr

benoit.mialet@edu.dsti.institute



School of Engineering

Learning, Evaluating, Testing

The Learning Objective

- ❑ A model learns by adjusting its parameters to minimize a loss function on training examples



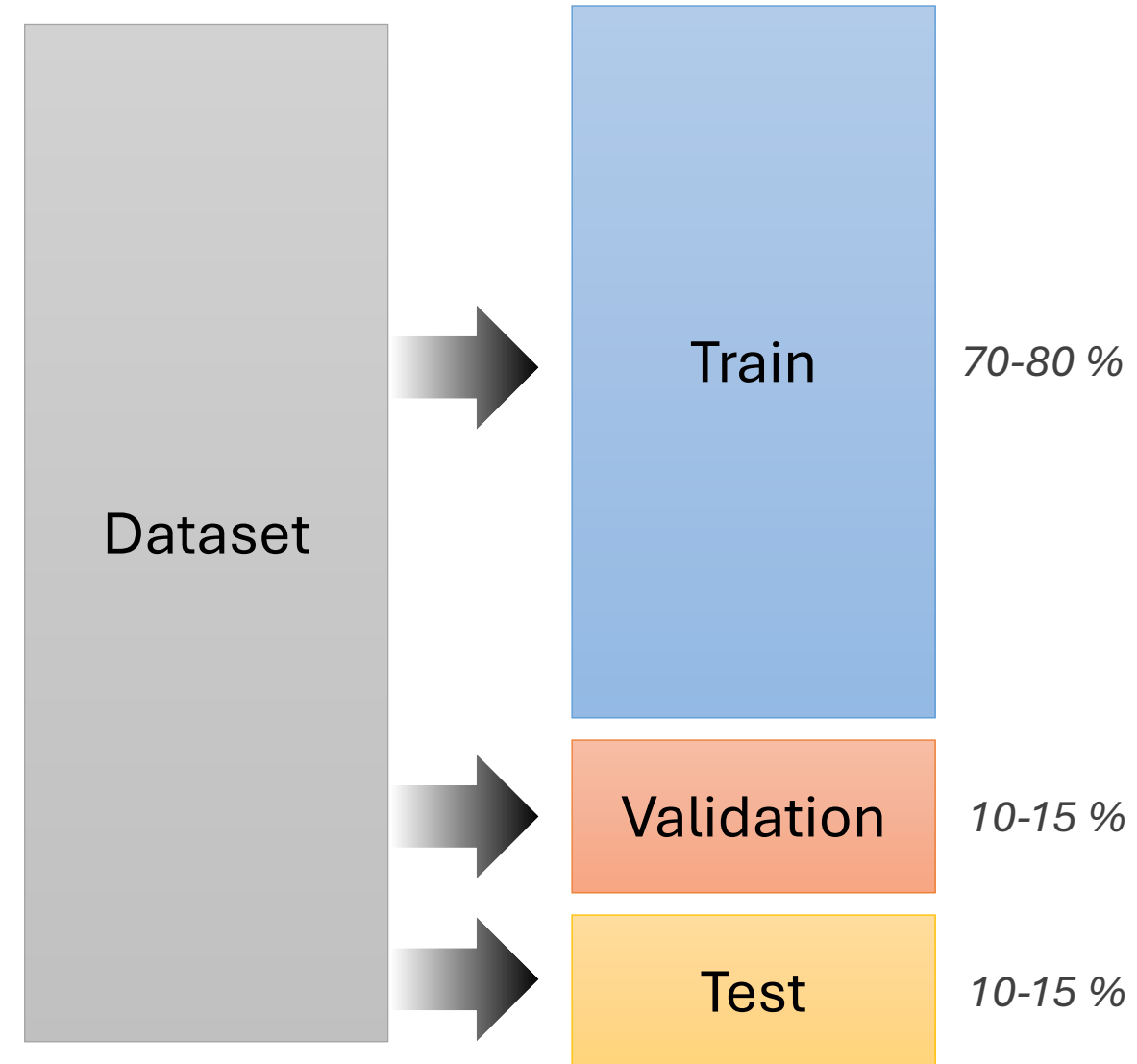
The Learning Objective

- ❑ A model learns by adjusting its parameters to minimize a loss function on training examples
- ❑ But to make a model useful we want to obtain low error not only on training data but also on new, unseen data
- ❑ **The real goal of training a model is to generalize well to unseen data**



Data Splitting

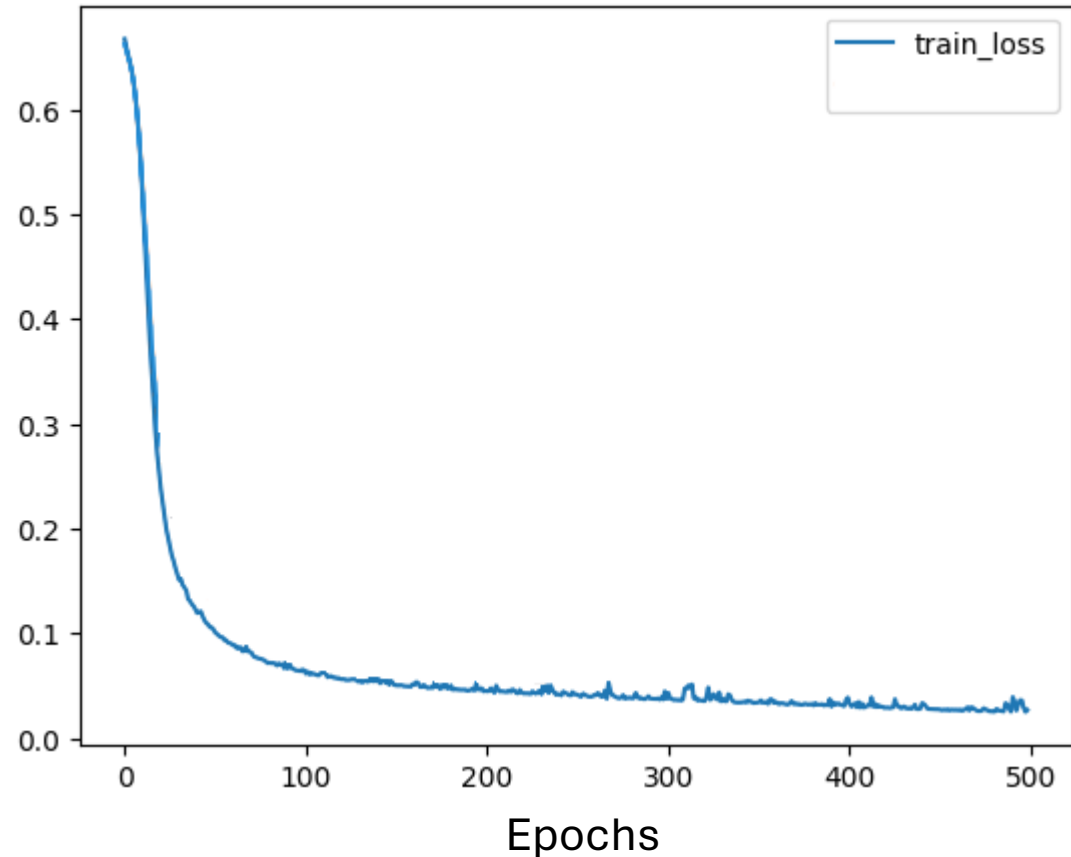
- ❑ Data Splitting is the key method to ensure model learns and generalizes well
- ❑ The basic ML setup requires to split available data into three **subsets** or “**splits**”:
 - **Training**
 - **Validation**
 - **Test**



Data Splitting

□ Train split

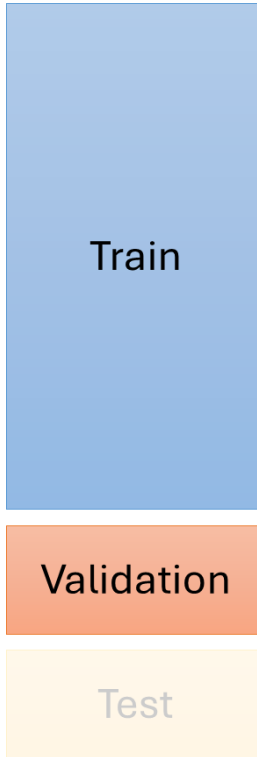
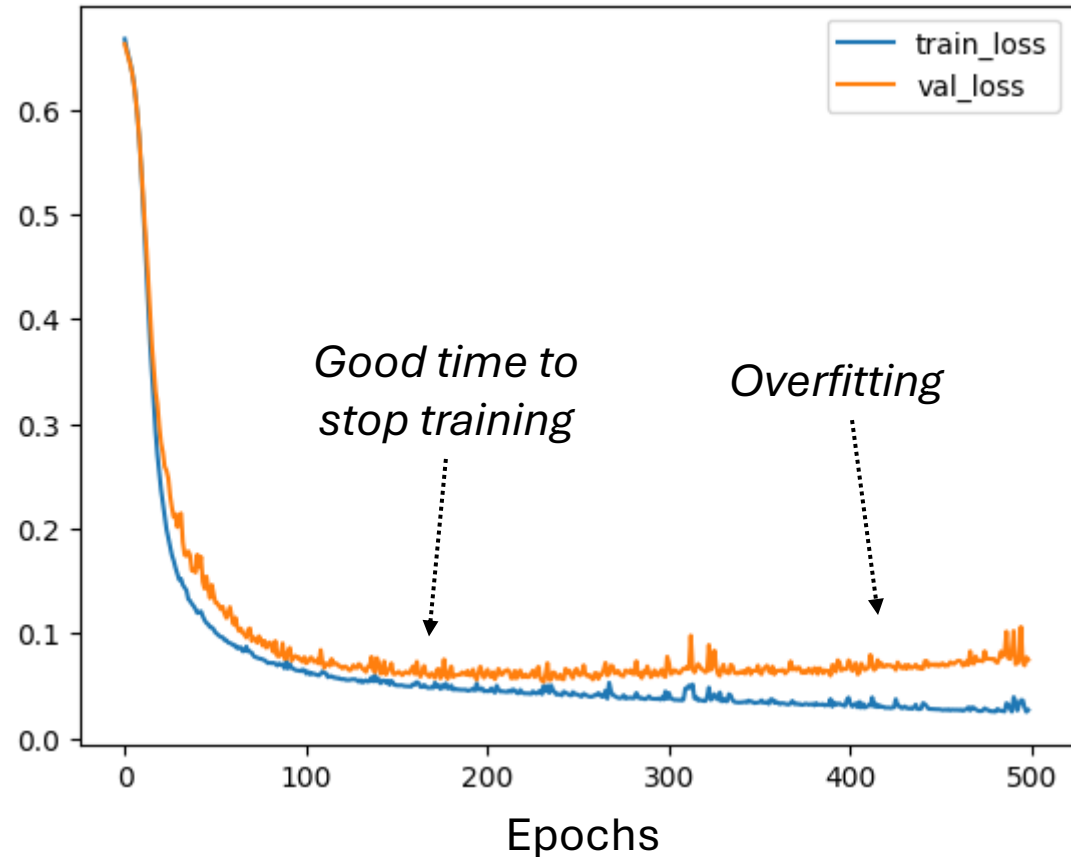
- Used to train the model (adjust parameters)
- During each epoch, model will learn the entire train data
- Loss and accuracy can be tracked to see training evolution across epochs



Data Splitting

❑ Validation split

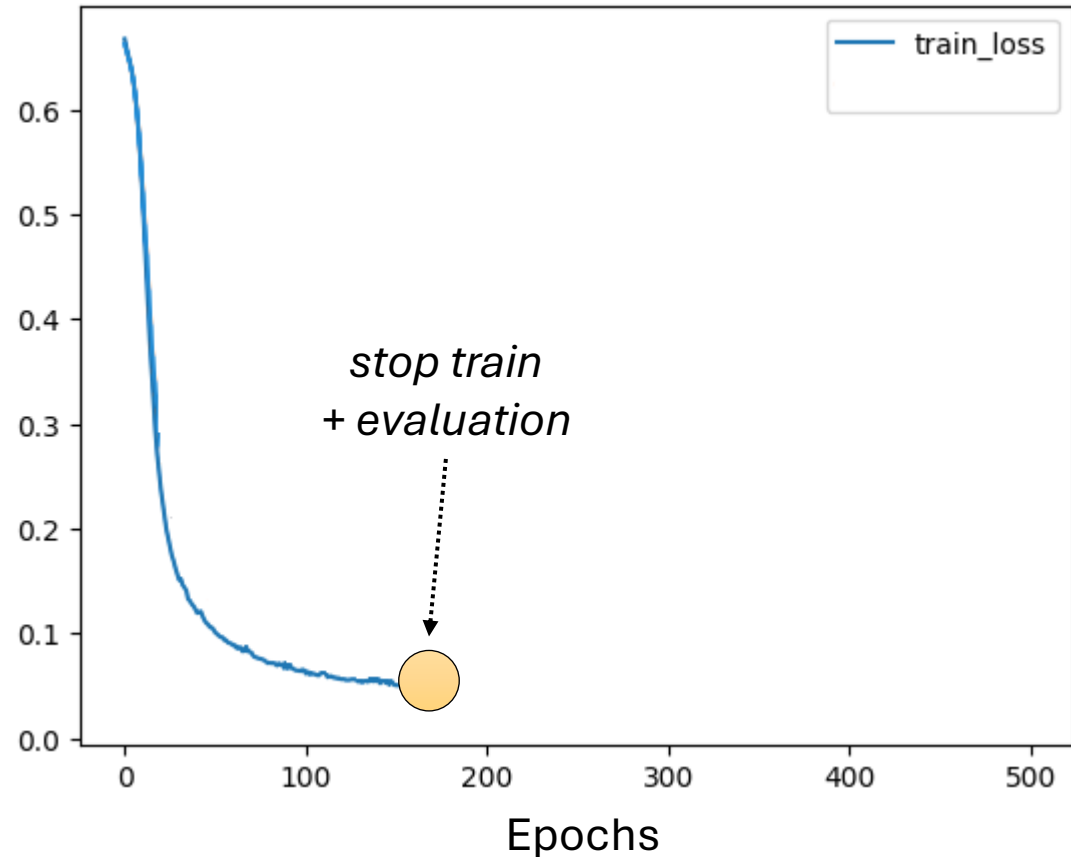
- Is unseen data: model does not learn on it (no parameter adjustment)
- **Is used to define training method**
- Loss and accuracy can be tracked during training, to detect **overfitting** (to be seen later)
- After training, validation loss is also used for **hyperparameter tuning**



Data Splitting

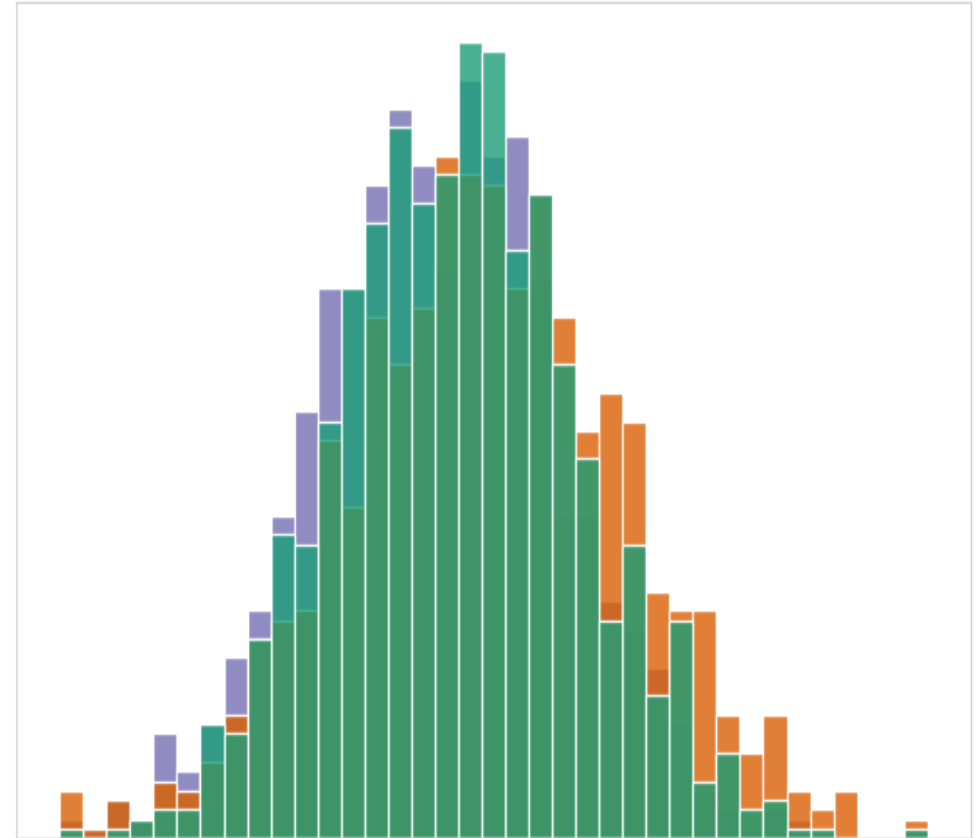
❑ Test split

- Is unseen data: model does not learn on it (no parameter adjustment)
- Simulates real-world data
- Is used **only once** after the training, to compute **evaluation metrics** (not loss), to evaluate model performance



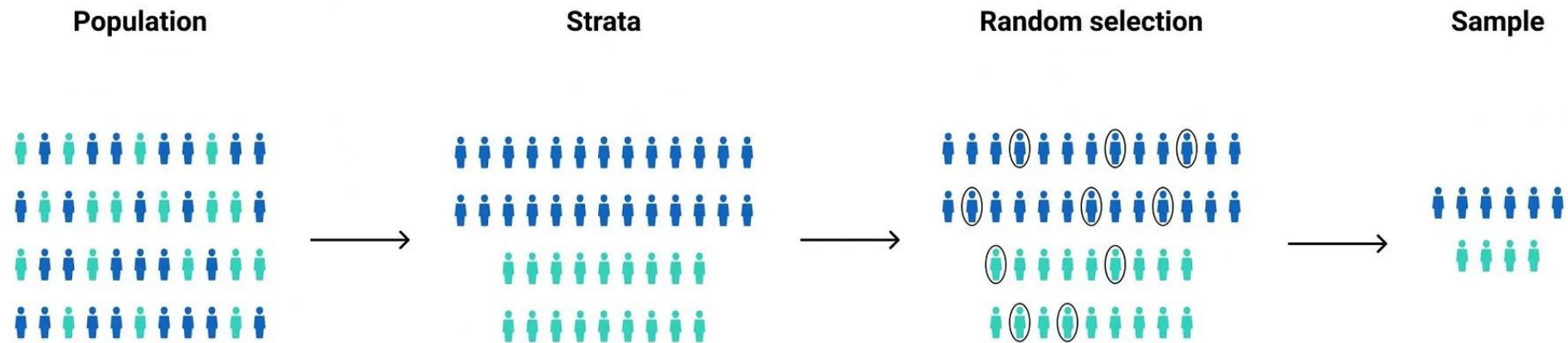
Data Splitting: gold rules

- ❑ **No duplicates** in data
 - -> No sample in several splits
- ❑ **All splits must have the same distribution**
 - For that, shuffling data prior to splitting is the best practice
 - If data has categories, sampling should be **stratified** : all categories in the same proportion across the splits



Stratification: two strategies

- ❑ Proportional stratified sampling
 - Each stratum (or category) is sampled in proportion to its size in the population
 - The sampling accurately reflects the population being studied
 - **Train:** often used, default strategy
 - **Test:** used to evaluate model on the population

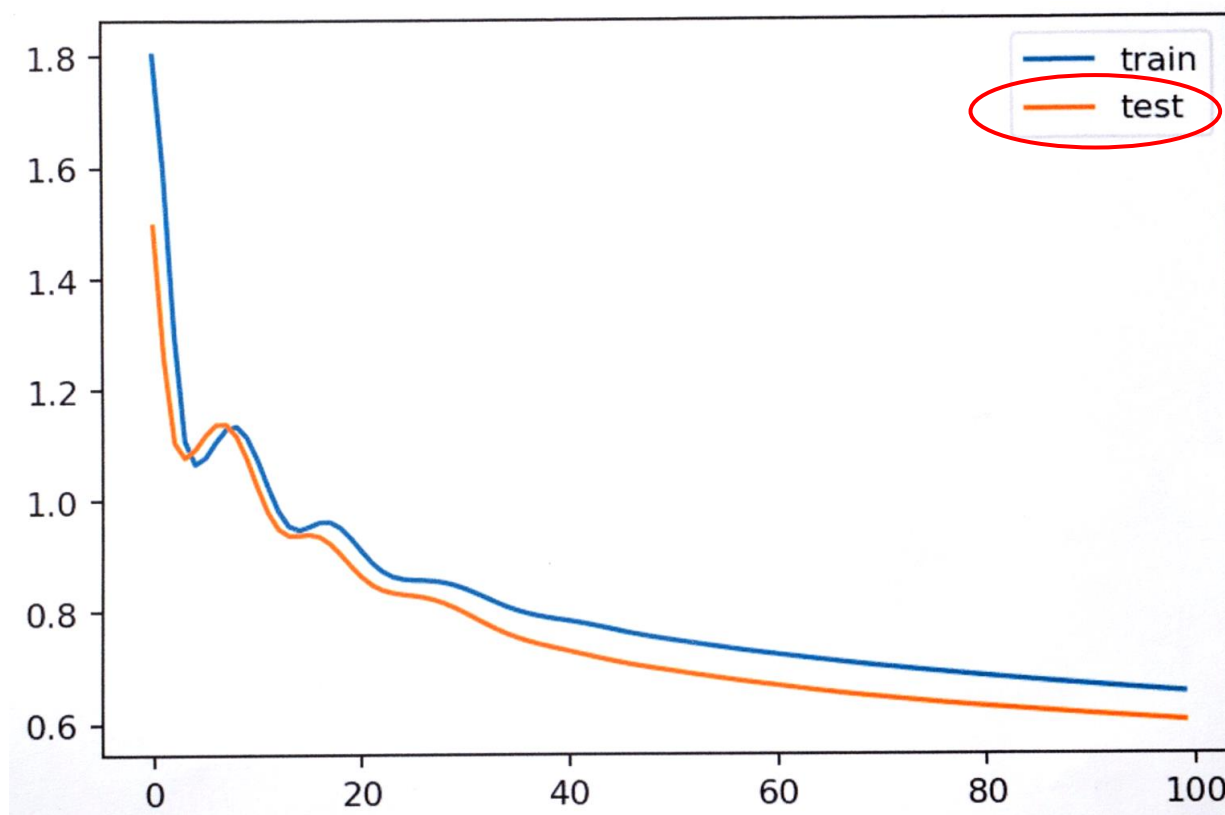


Stratification: two strategies

- ❑ Equal (or balanced) stratified sampling
 - Each stratum have the same number of samples, regardless of its size in the population
 - Ensures that each stratum is equally represented
 - **Train:** sometimes used, when it is important that the model performs well on every stratum
 - **Test:** used to evaluate model on each stratum



Confusing ? Why “test” and not “validation”



Data Splitting



A “**test**” set is sometime used to track validation loss during training. This set should be called “**validation**” instead.

Inversely, if only 2 splits are used, the second one should be called “**test**” set, because it is used for **final evaluation**. If a validation set is needed, it can be sub-sampled from the training set.

Calling a final evaluation split “**validation**” is misleading. However, in practice people often misuse the term “validation” to mean “test”.

Be aware of these confusions!

Underfitting and Overfitting

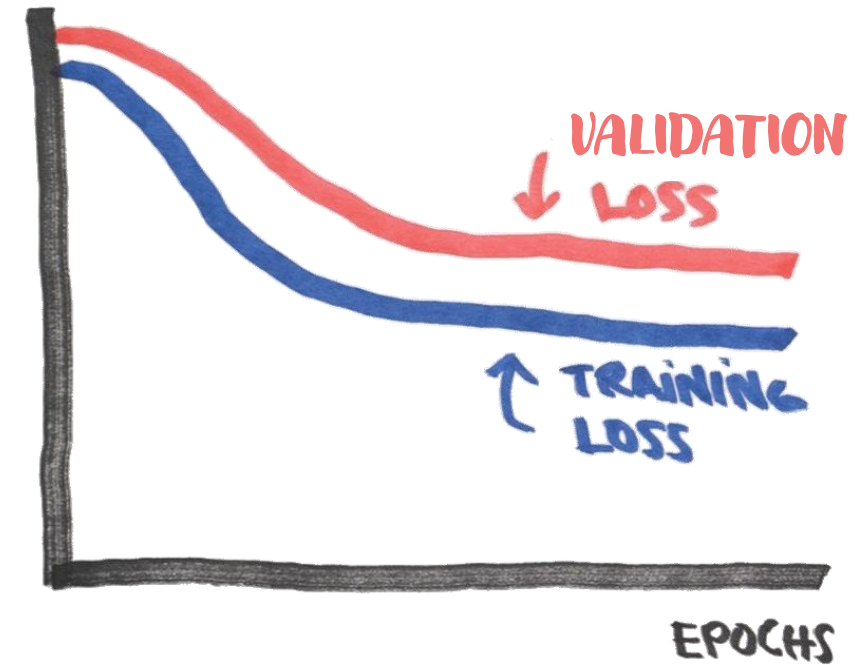
What is underfitting?

- ❑ **Underfitting** occurs when the model fails to learn from the training data. As a result, it also **fails to generalize to unseen data**
- ❑ Underfitting causes can be multiple:
 - The model is too simple to capture the underlying patterns in the data (insufficient complexity)
 - The model is not trained enough
 - Not enough data
 - Not enough epochs
 - Too little training steps...



How to detect underfitting?

- ❑ This bad performance can be detected through evaluation metrics
- ❑ High training error: the model doesn't fit the training data well
- ❑ High validation error: it doesn't generalize either



What is overfitting?

- ❑ **Overfitting** occurs when a model learns the training data too well, including its noise and outliers
- ❑ As a result, it performs well on the training set but **fails to generalize to unseen data**

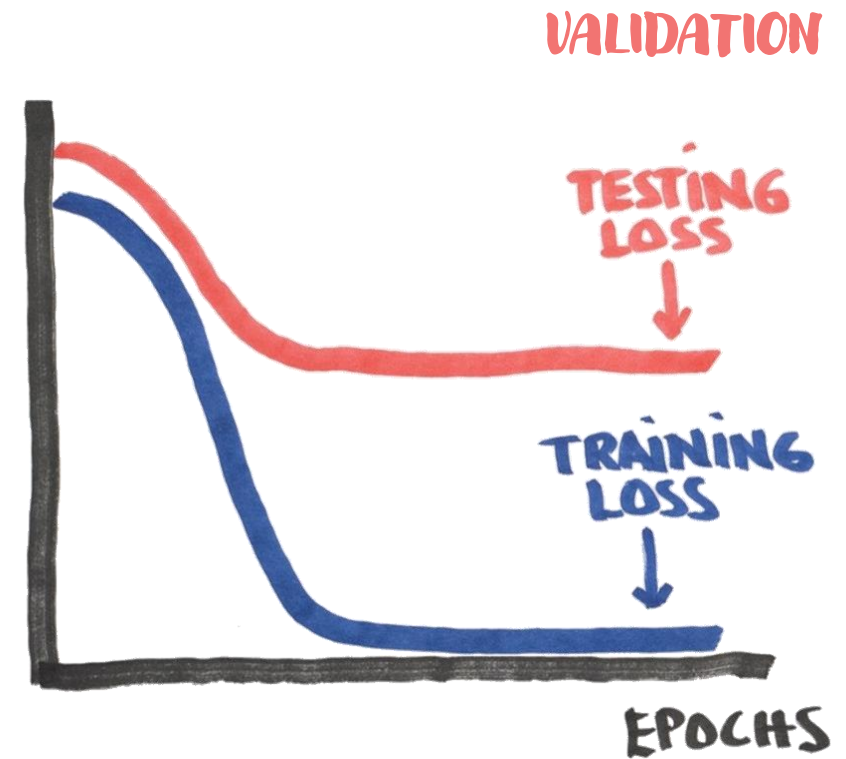
What is overfitting?

- ❑ **Overfitting** occurs when a model learns the training data too well, including its noise and outliers
- ❑ As a result, it performs well on the training set but **fails to generalize to unseen data**
- ❑ Main overfitting causes:
 - The model is too complex for the problem and memorizes the data instead of learning meaningful patterns
 - The model is trained for too many epochs



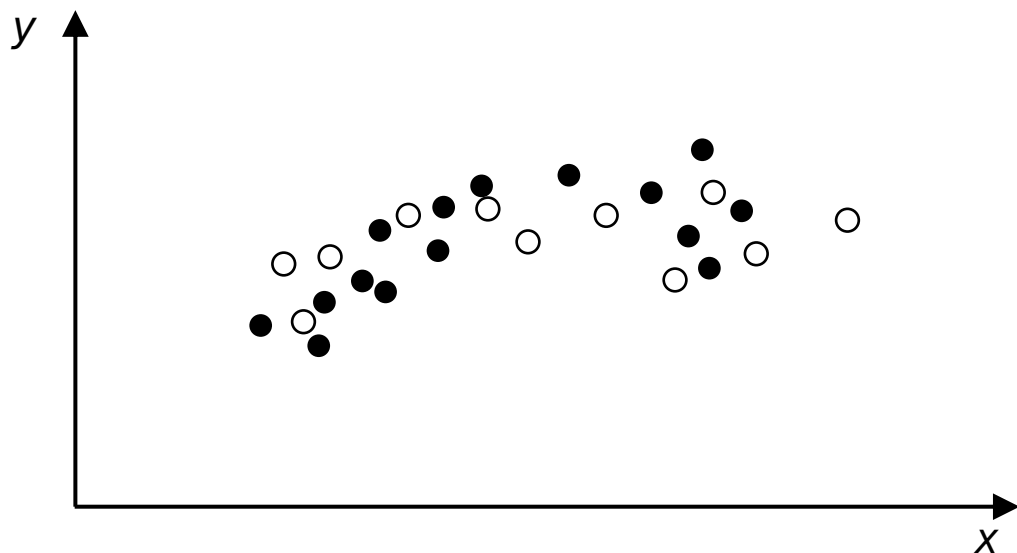
How to detect overfitting?

- ❑ This bad performance can be detected through evaluation metrics
- ❑ Very low training error
- ❑ High Validation error
- ❑ Performance gap grows with epochs

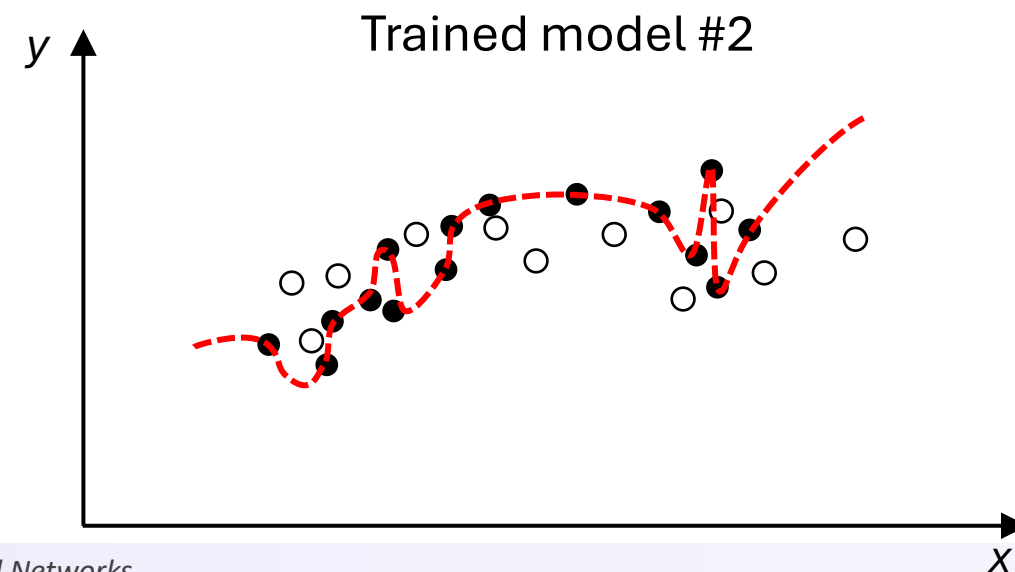
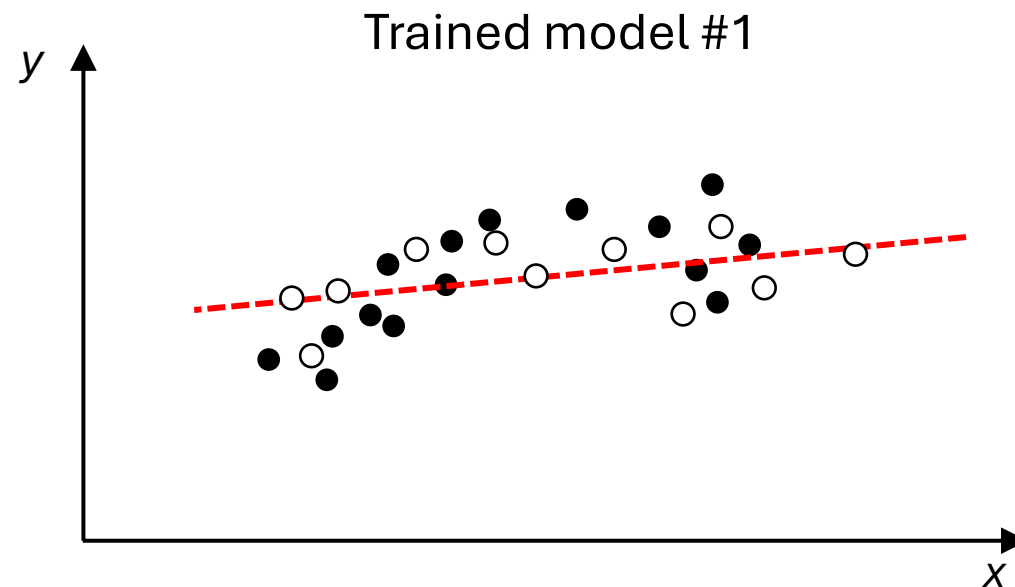


Bias Variance Trade-off

□ Let's imagine this situation...



- Training data
- Testing data

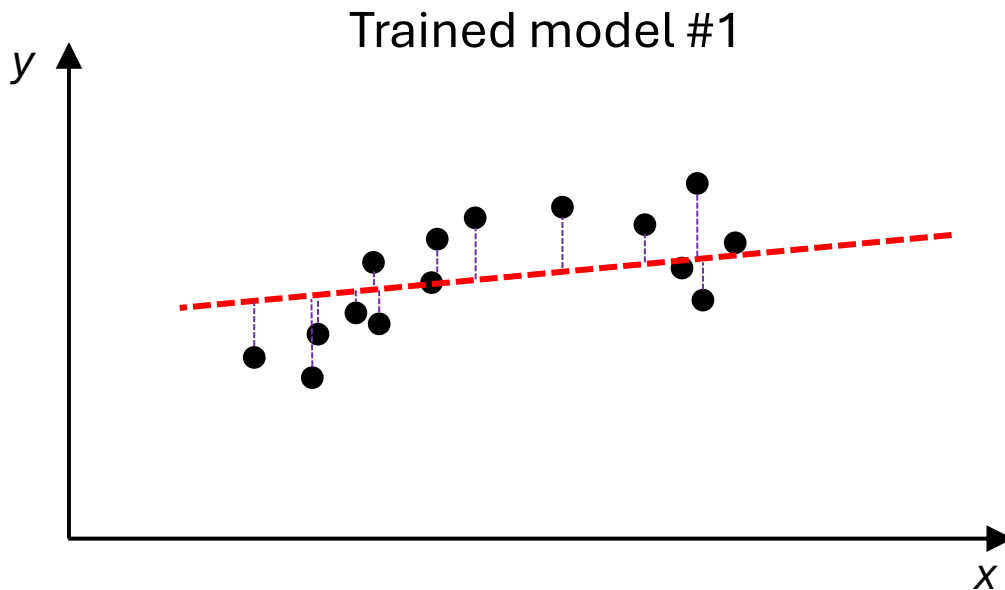


Bias Variance Trade-off

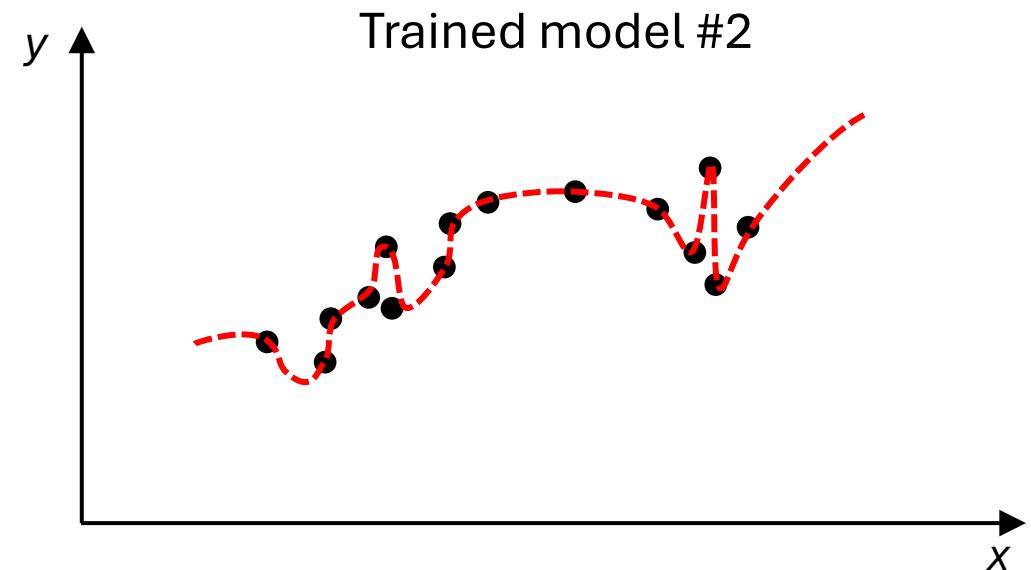
- Let's compare error of both models on train dataset

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

➡ $RMSE$ is quite high



➡ $RMSE = 0$ Perfect!

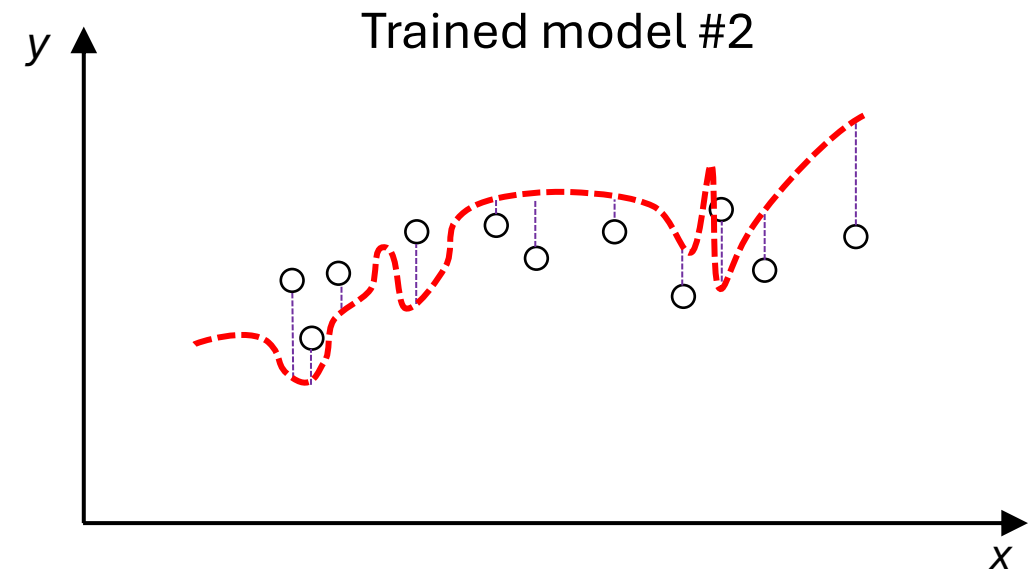
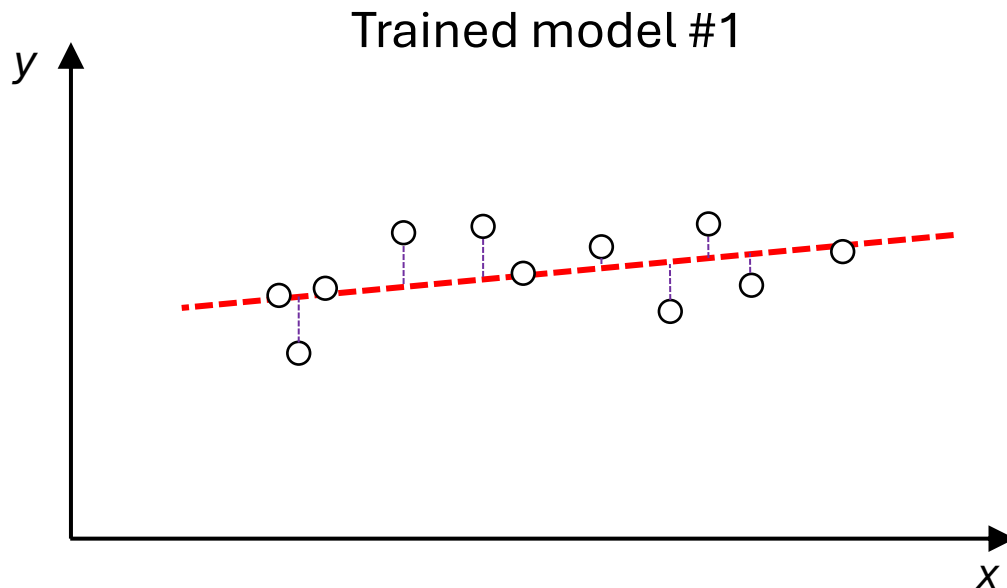


Bias Variance Trade-off

- Let's compare error of both models on test dataset

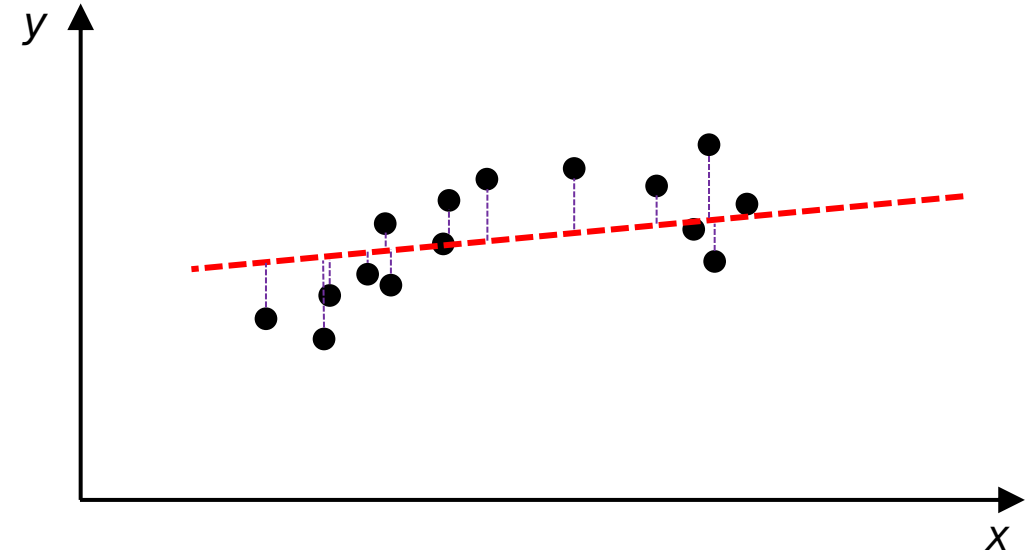
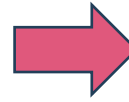
$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

➡ $RMSE_2 > RMSE_1$ Actually, model #1 is performing better on unseen data!

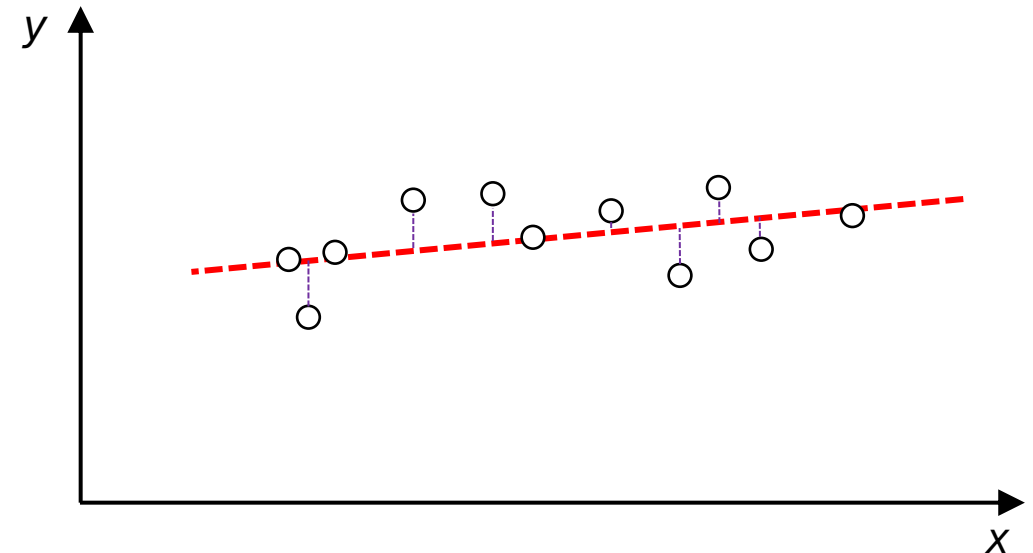
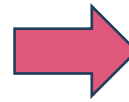


Bias Variance Trade-off

- ❑ **Bias** measures how far off predictions are from the true values due to overly simple assumptions

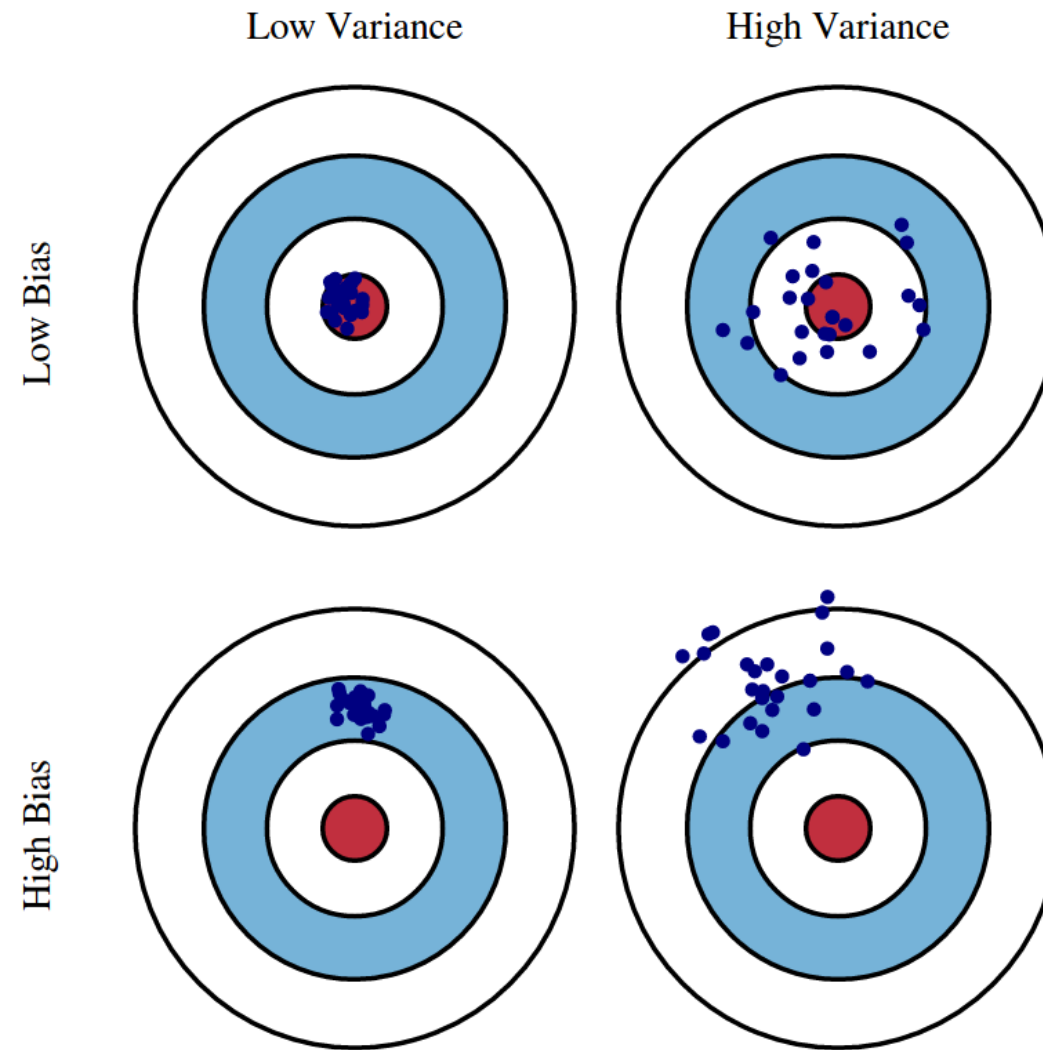


- ❑ **Variance** measures how much a model's predictions would vary if we trained it on different datasets



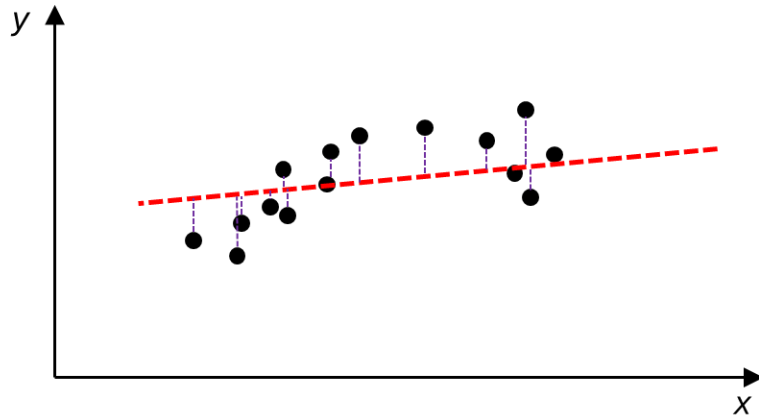
- Training data
- Testing data

Bias Variance Trade-off



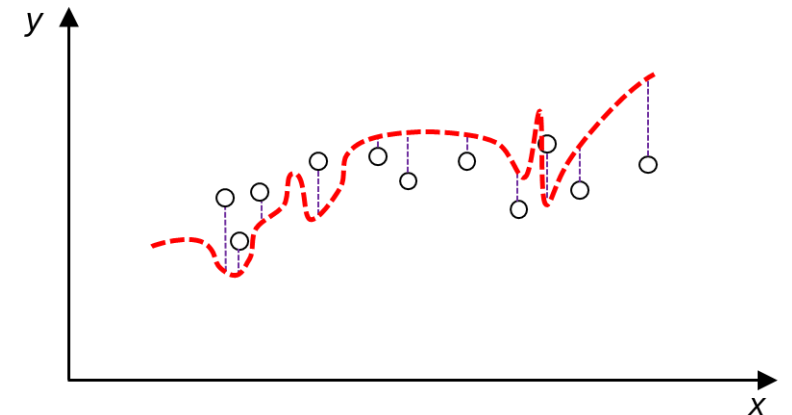
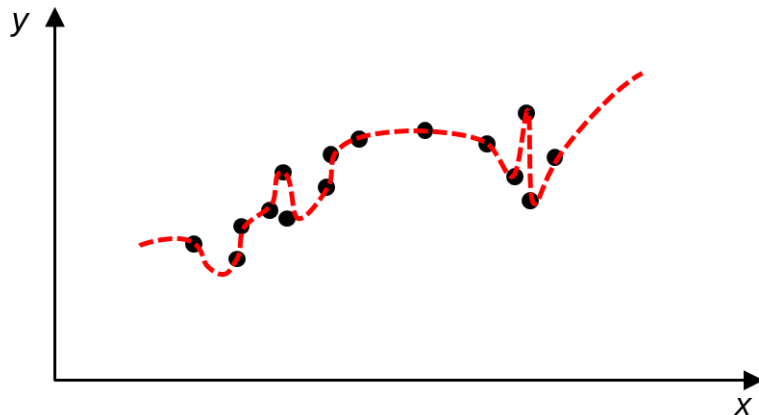
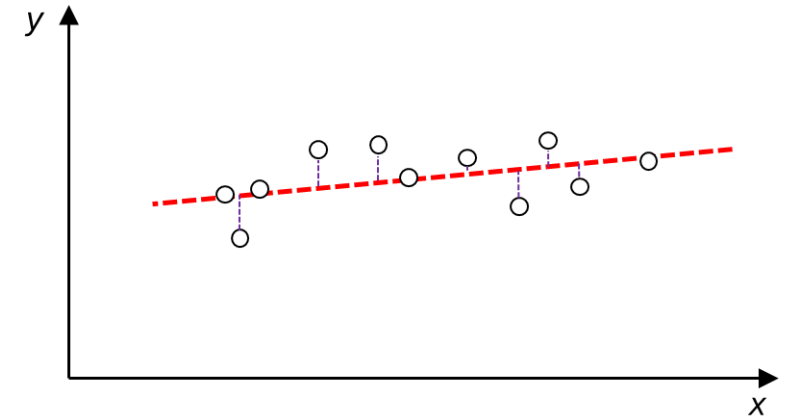
<https://scott.fortmann-roe.com/docs/BiasVariance.html>

Bias Variance Trade-off



How could we qualify
bias and variance of
both models?

What do you think?

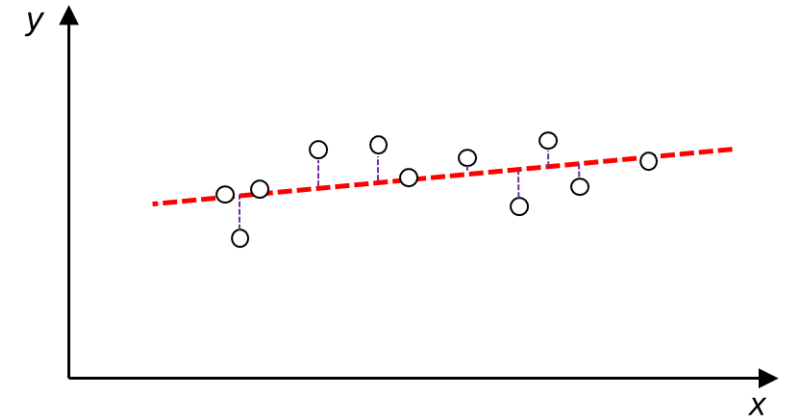
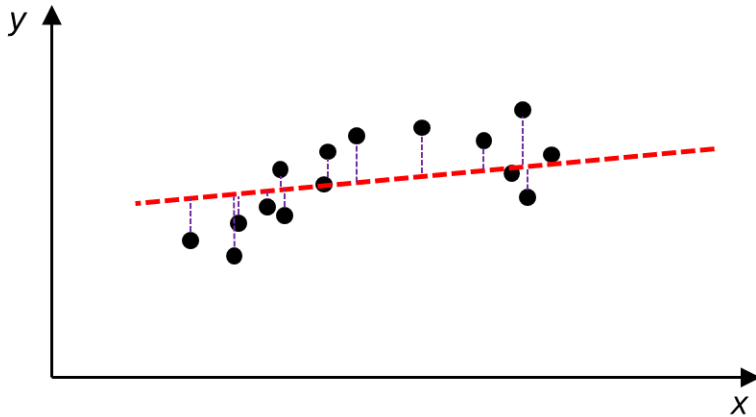


Bias Variance Trade-off

□ Model #1

- Rather high bias
- Rather low variance

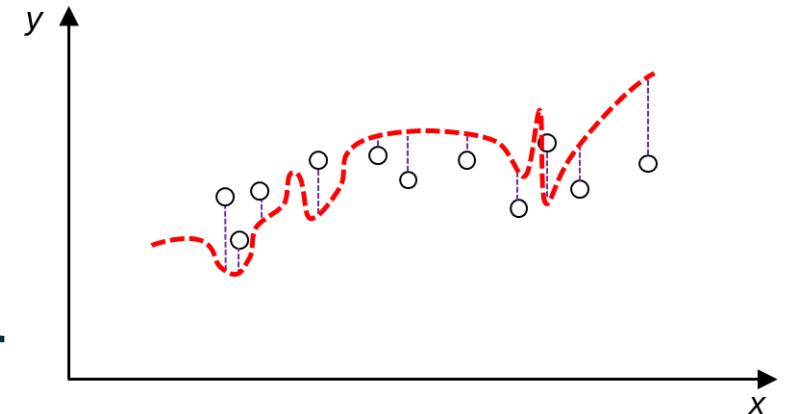
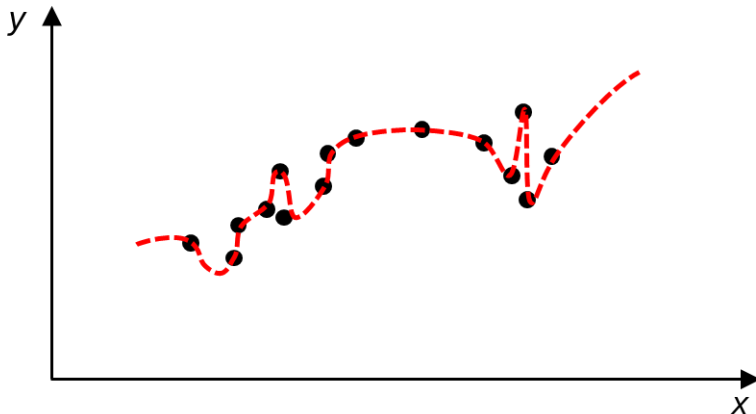
➡ **Predictable model**



□ Model #2

- Low bias
- High variance

➡ **Unpredictable model**



Bias Variance Trade-off

- ❑ The total model error comes from two components:

$$\textit{Total Error} = \textit{Bias}^2 + \textit{Variance} + \textit{Irreducible Error}$$

Noise

- ❑ **We can't eliminate both:** reducing one often increases the other
- ❑ However, there are some ways of mitigation

Bias Variance Trade-off: mitigation

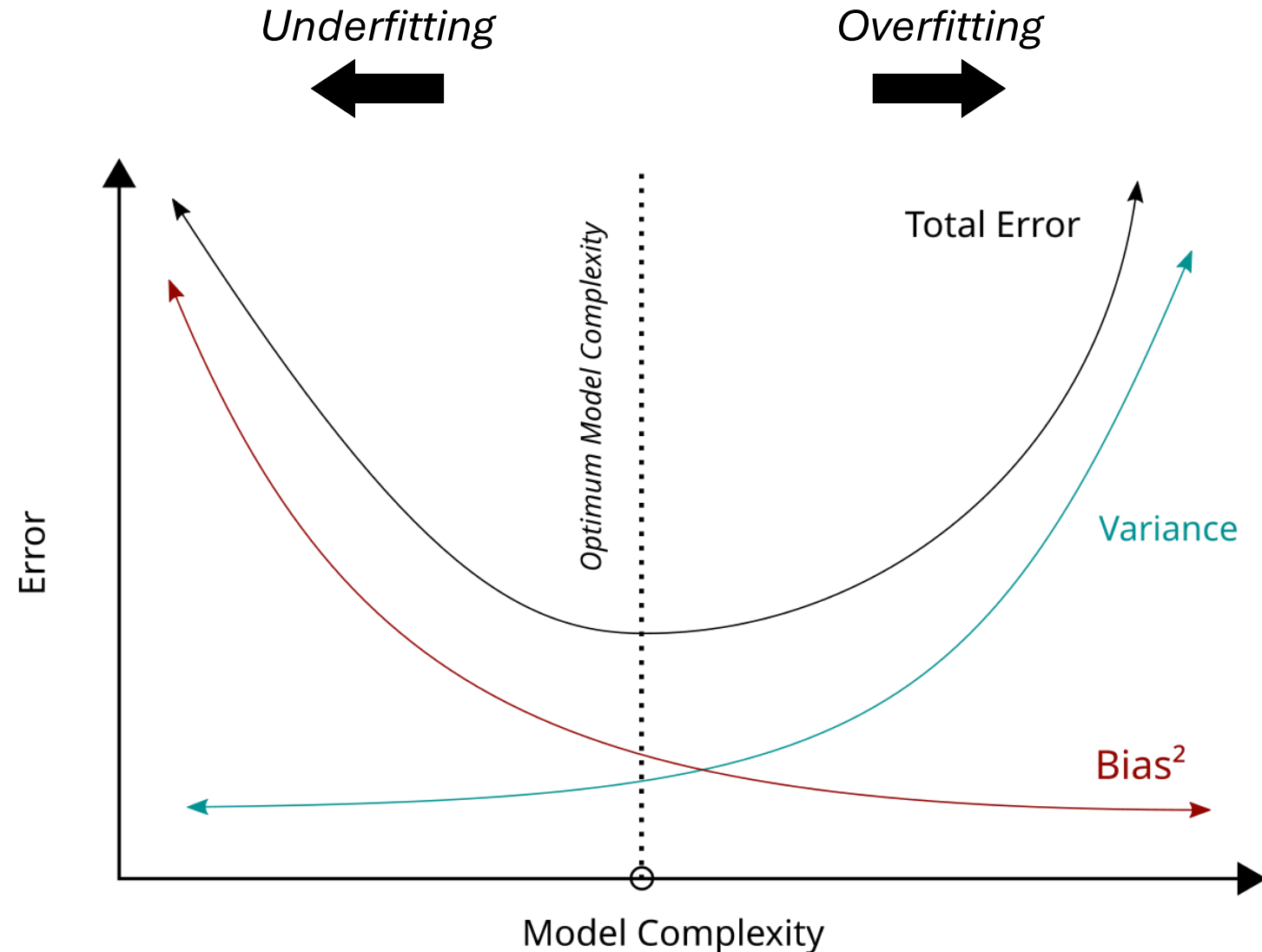
1 First goal is to find the optimal **model complexity** that minimizes total error

□ Increasing model complexity:

- - Bias
- + Variance

□ Decreasing model complexity:

- + Bias
- - Variance

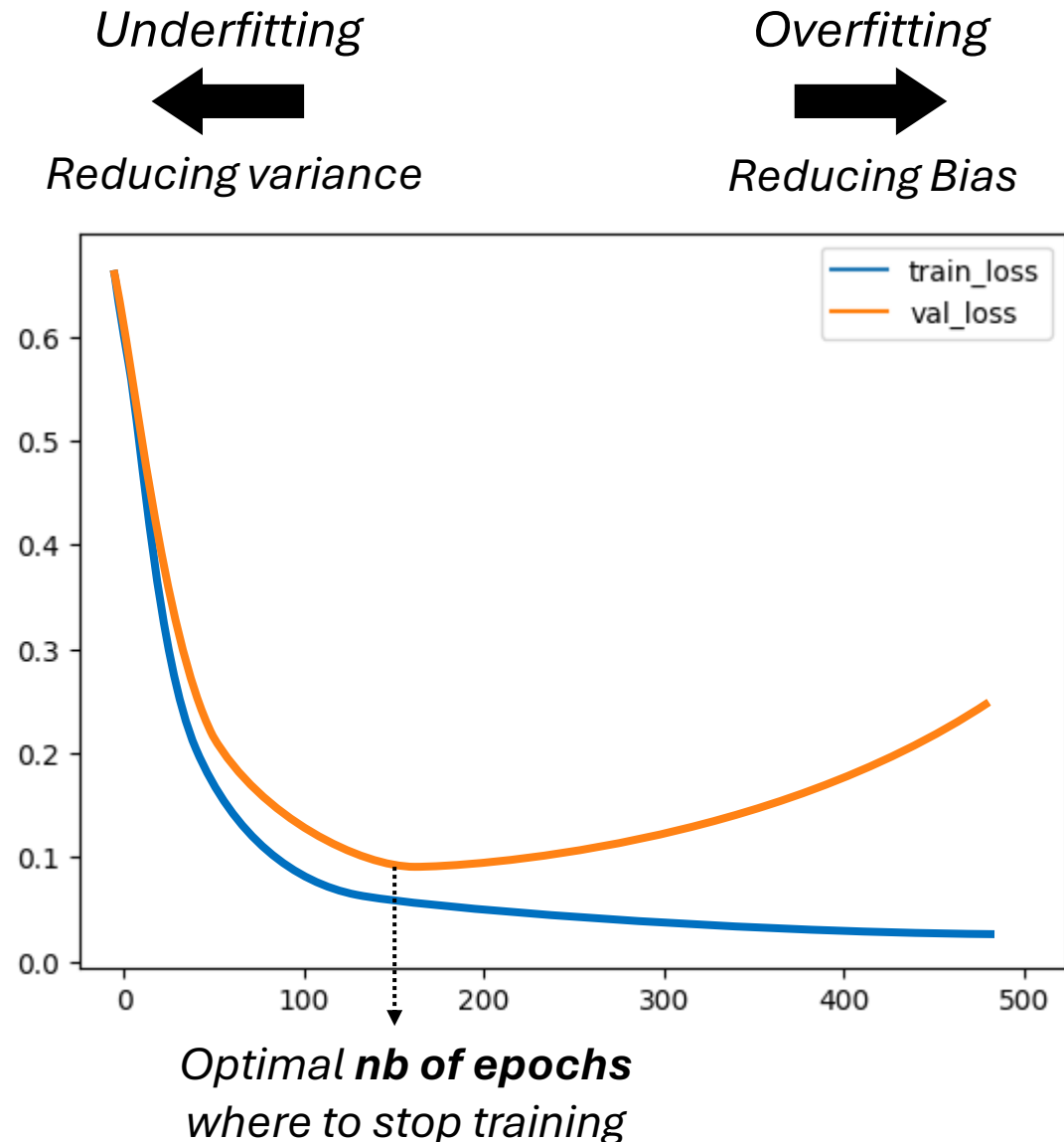


<https://scott.fortmann-roe.com/docs/BiasVariance.html>

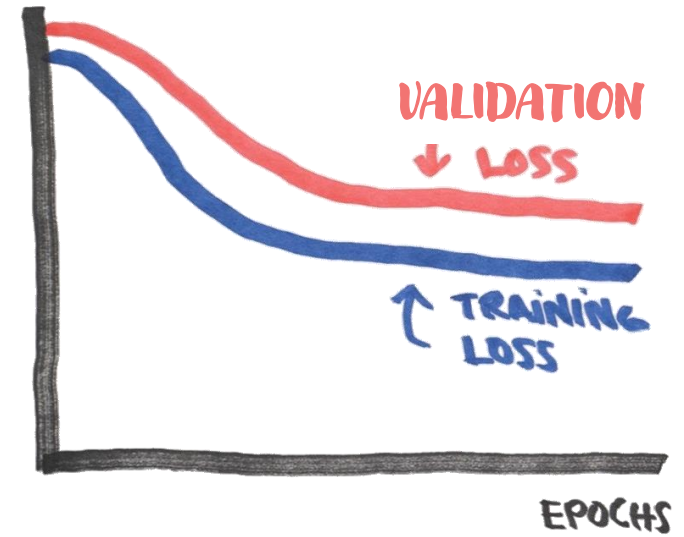
Bias Variance Trade-off: mitigation

2 Second goal is to find the optimal amount of training with the same data

- ❑ **Early Stopping** during training is the best way to achieve this
- ❑ **Stop training when validation loss stops improving**, even if training loss keeps decreasing
- ❑ Can be done with loss or any evaluation metric as well

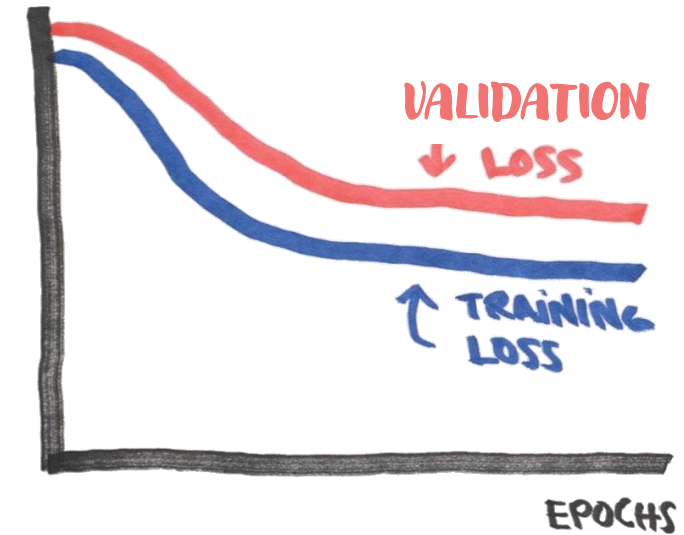


How to avoid underfitting?

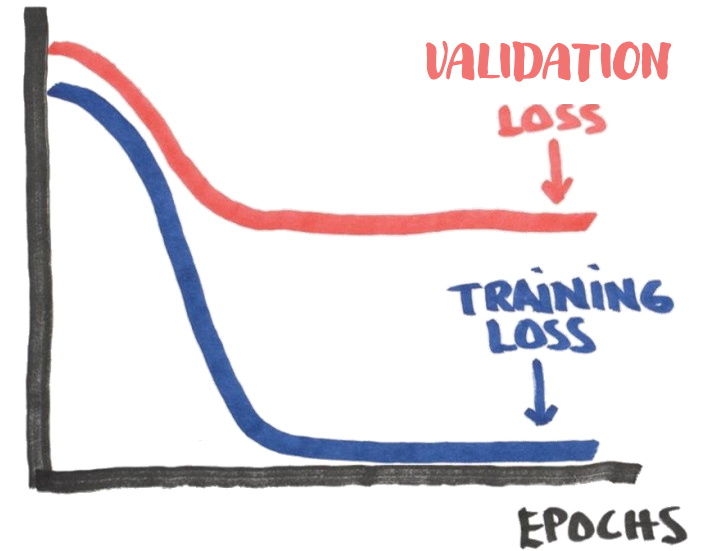


How to avoid underfitting?

- ❑ Increase model complexity (add more neurons or layers if appropriate)
- ❑ Train on more epochs
- ❑ Add data features (independent variables)
- ❑ Add more data
- ❑ Improve data quality (diversity, correctly labelled, remove redundancy)

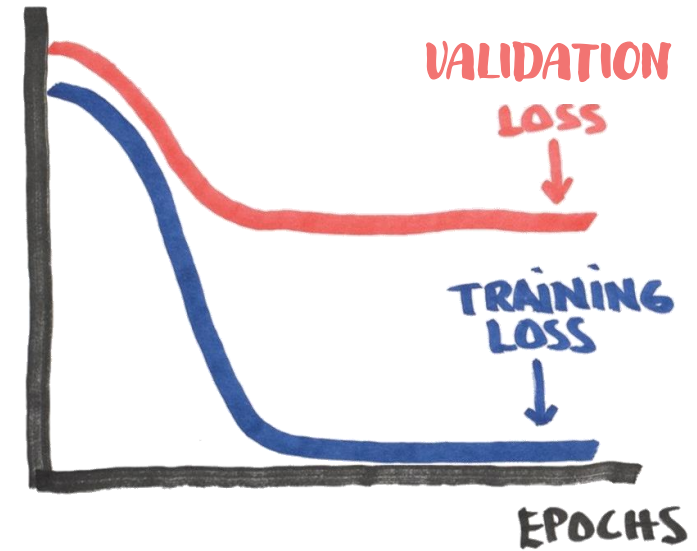


How to avoid overfitting?



How to avoid overfitting?

- ❑ Reduce model complexity
- ❑ Early Stopping
- ❑ Remove irrelevant features (selection)
- ❑ Add more data
- ❑ Regularisation

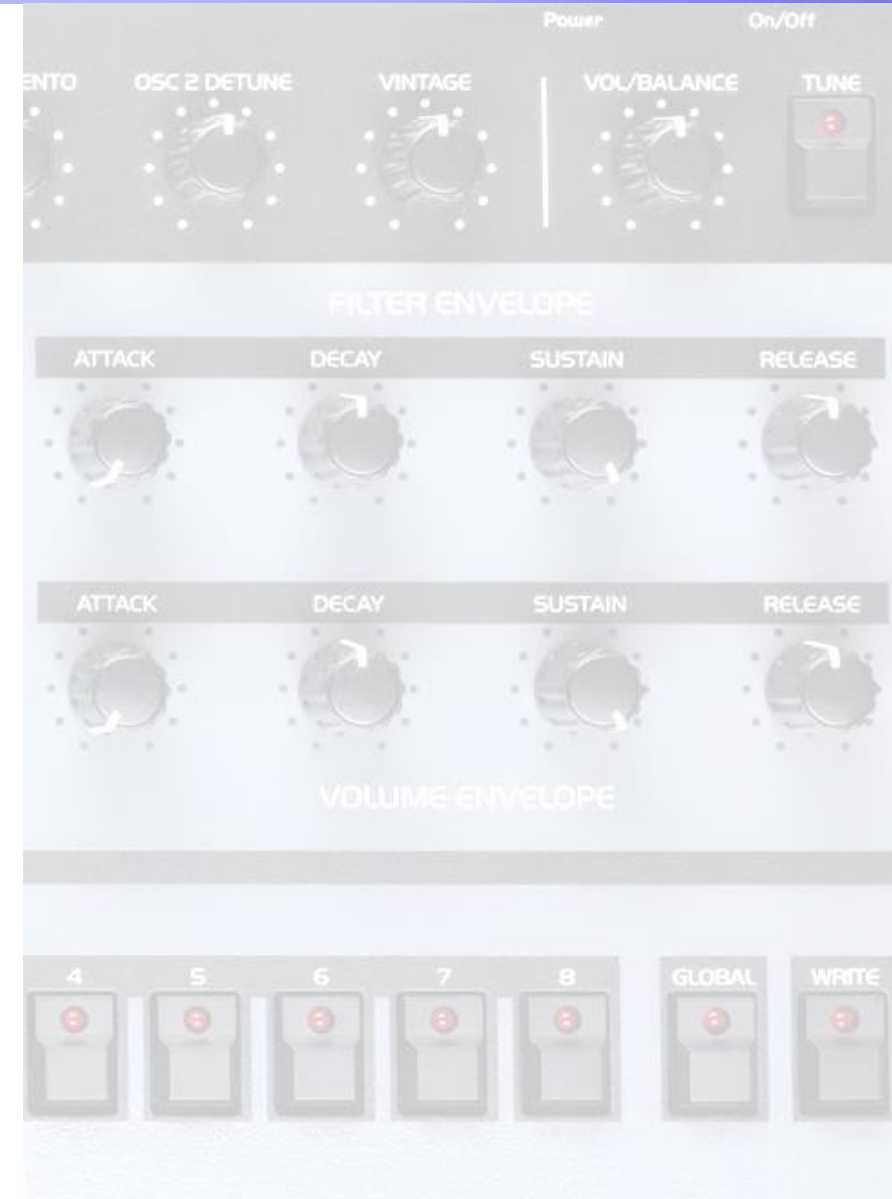


Hyperparameter tuning



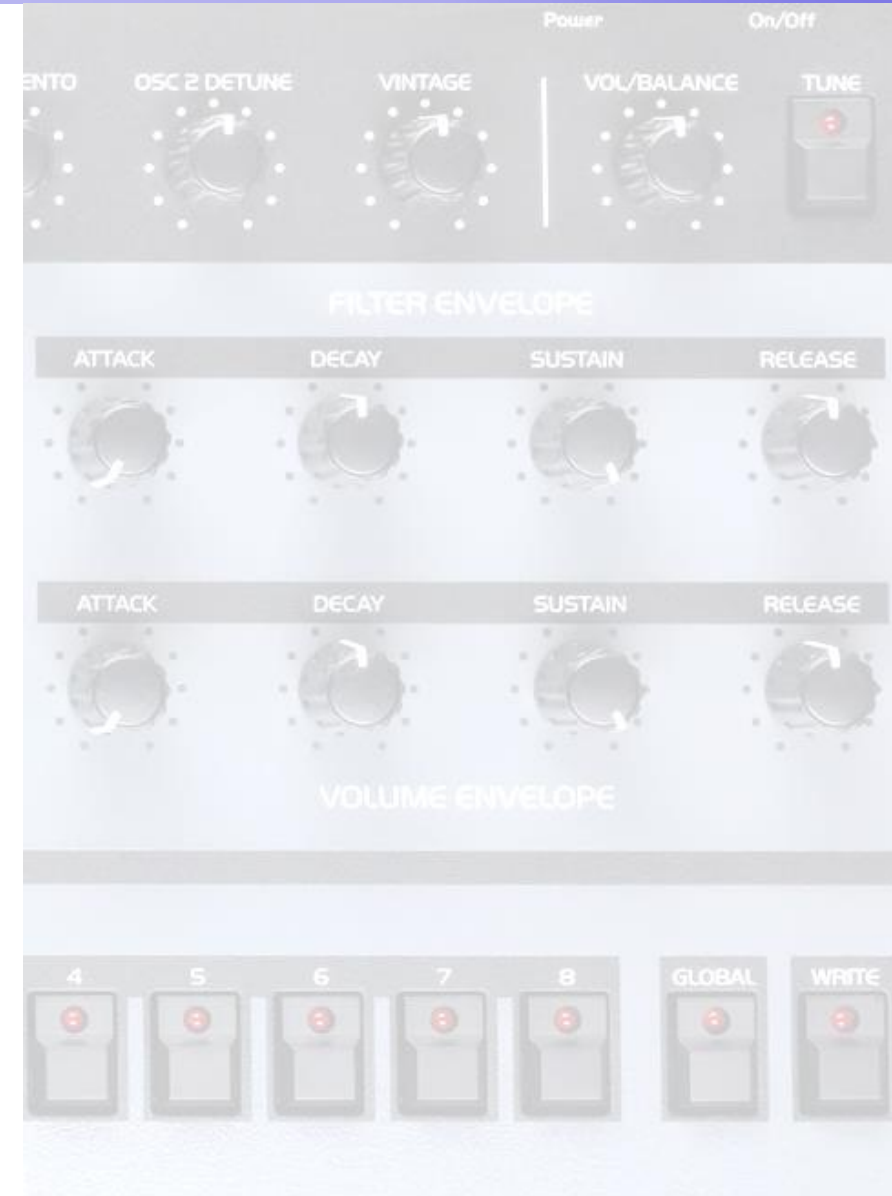
Hyperparameters

- ❑ What are Hyperparameters?
 - Variables that control the learning process but are **not learned from data**
 - They have to be set before the model's training



Hyperparameters

- ❑ What are Hyperparameters?
 - Variables that control the learning process but are **not learned from data**
 - They have to be set before the model's training
- ❑ Why tuning Hyperparameters?
 - Different settings give very different performance (after training)
 - There is **no universal best set**. It will depend on dataset, model, and task
 - Tuning balances between underfitting and overfitting
 - Tuning can reduce training time (e.g. good learning rate)
- ❑ How to tune Hyperparameters?
 - Try different settings and watch Loss scores on **validation split**



Hyperparameters (most common ones)

Training-related

Learning rate
[0; 1]

Number of epochs
[1; ∞]

Batch size
[1; ∞]

Hyperparameters (most common ones)

Training-related

Learning rate
[0; 1]

Number of epochs
[1; ∞]

Batch size
[1; ∞]

Architecture-related

**Number of hidden
layers and
neurons**

**Activation
functions**

Hyperparameters (most common ones)

Training-related

Learning rate
[0; 1]

Number of epochs
[1; ∞]

Batch size
[1; ∞]

Architecture-related

**Number of hidden
layers and
neurons**

**Activation
functions**

Regularisation-related

**L2 Regularization
(weight decay)**
→ α
[0; 1]

Dropout Rate
[0; 1]

Hyperparameter tuning approaches

❑ Grid Search

- Try all combinations from a predefined set of values
- Expensive, scales poorly

Hyperparameter tuning approaches

❑ Grid Search

- Try all combinations from a predefined set of values
- Expensive, scales poorly

❑ Random Search

- Sample hyperparameters randomly, given a range
- Often finds good results faster than grid search

Hyperparameter tuning approaches

❑ Grid Search

- Try all combinations from a predefined set of values
- Expensive, scales poorly

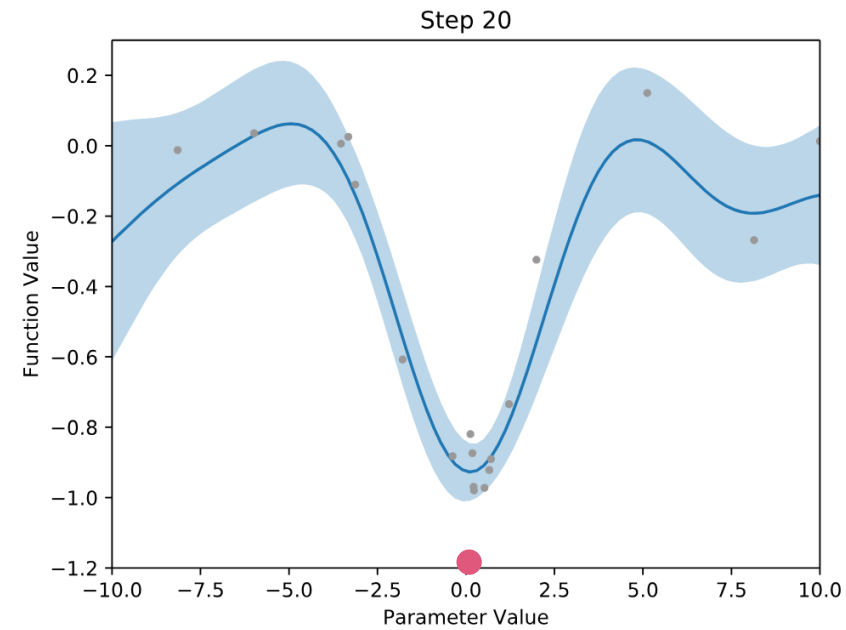
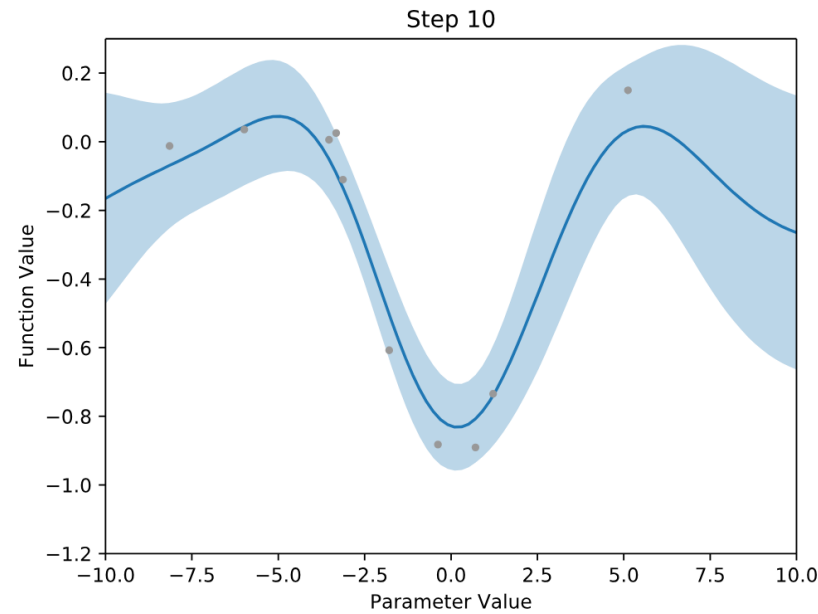
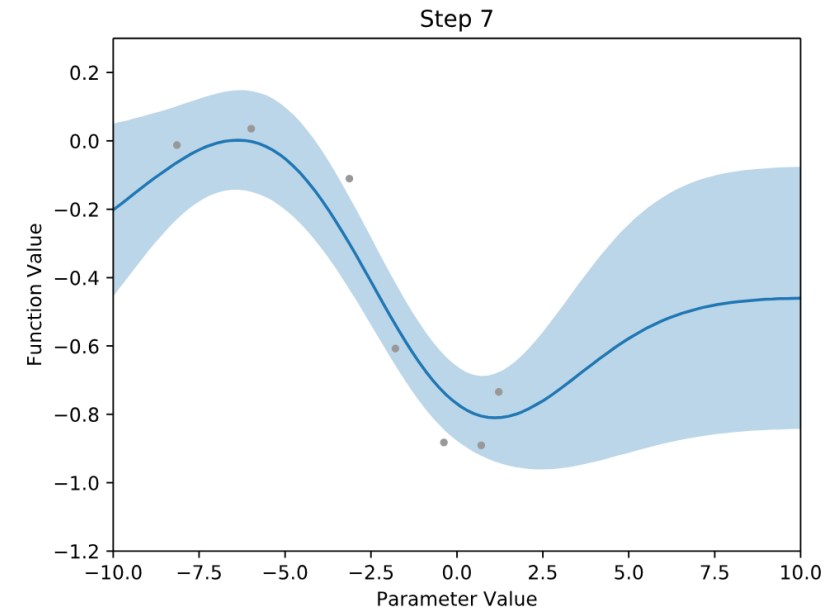
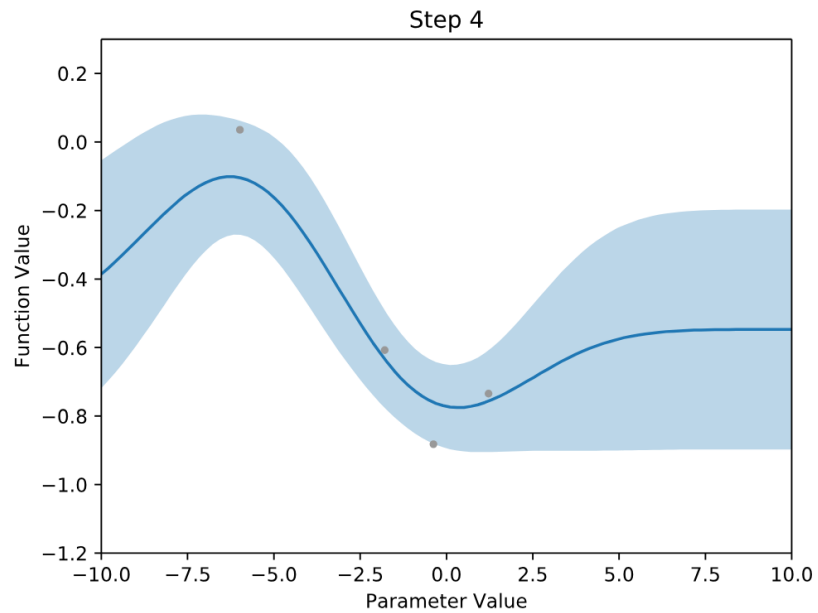
❑ Random Search

- Sample hyperparameters randomly, given a range
- Often finds good results faster than grid search

❑ Bayesian Optimization

- Bayesian optimization uses a model to predict $Loss = f(value)$
- It balances trying new regions (exploration) and focusing on the best regions found so far (exploitation)
- More efficient, state-of-the-art for tuning

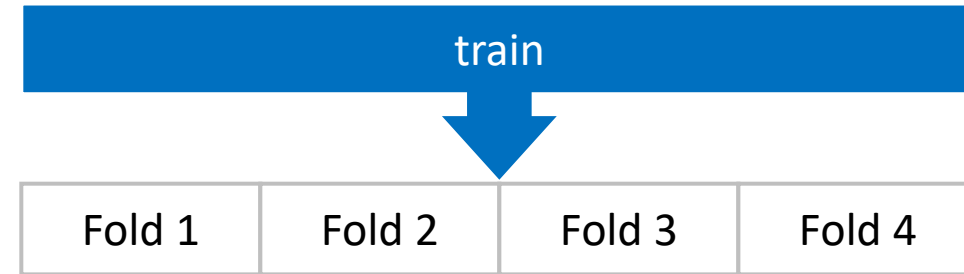
Bayesian Optimization (20 trials example)



<https://brendanhasz.github.io/2019/03/28/hyperparameter-optimization.html>

Hyperparameter tuning approaches

- ❑ **K-fold Cross validation** is a technique to evaluate precisely a model's performance
- ❑ Principle: for each setting, train K times the model with different subsets of available data, average results across folds
- ❑ It is used for hyperparameter tuning by comparing then picking the setting with best performance
- ❑ It is accurate but computationally expensive for neural networks
- ❑ It is still relevant nowadays for small models & small datasets



Example of 4-fold cross validation for learning rate = 0,0005:

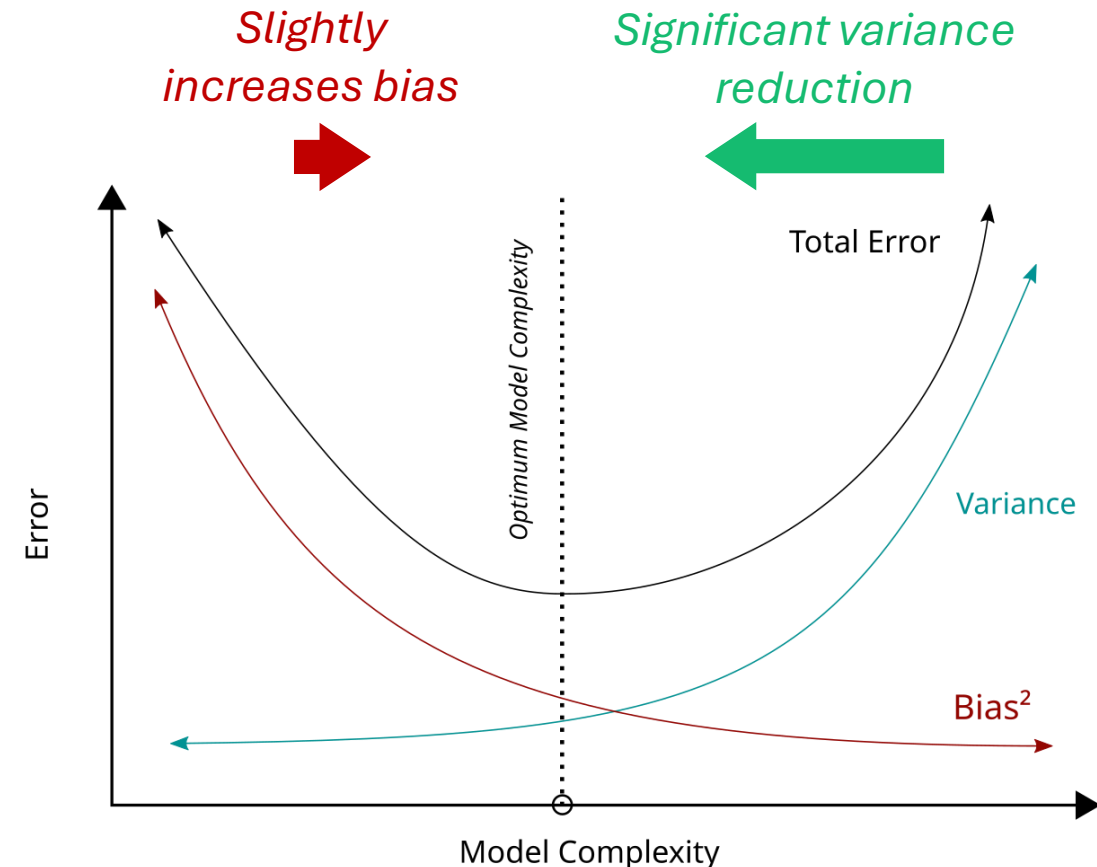
train	train	train	validation	→	87.5 %
train	train	validation	train	→	88.1 %
train	validation	train	train	→	88.3 %
validation	train	train	train	→	87.6 %
					↓
					Mean accuracy: 87.9 %

Regularisation Methods

What is Regularisation?

- ❑ **Regularisation** is a set of techniques used to **reduce overfitting** and improve **generalisation** by constraining the learning process of a model
- ❑ But it can also apply to the control of training data (**feature selection**)

Generalisation:
better performance
on unseen data!



1 L_1 (Lasso Regression)

- ❑ L_1 introduces a weight penalty in Loss calculation
- ❑ L_1 shrinks small (less important) weights **to zero** (it introduces sparsity)
- ❑ L_1 can be used as a **feature selection** method (with random forests, linear regression...)
- ❑ L_1 is less common in neural networks because less beneficial than L_2

$$L_1 = \text{Loss function} + \underbrace{\lambda \sum_{i=1}^n |w_i|}_{\substack{\text{Penalty term} \\ \text{Adds a bias to} \\ \text{avoid overfitting}}}$$

- λ is a parameter that determines the strength of the regularization
- $\lambda = 0$: no regularisation
- λ can be tuned with hyperparameter tuning

2

L_2 (Ridge Regression)

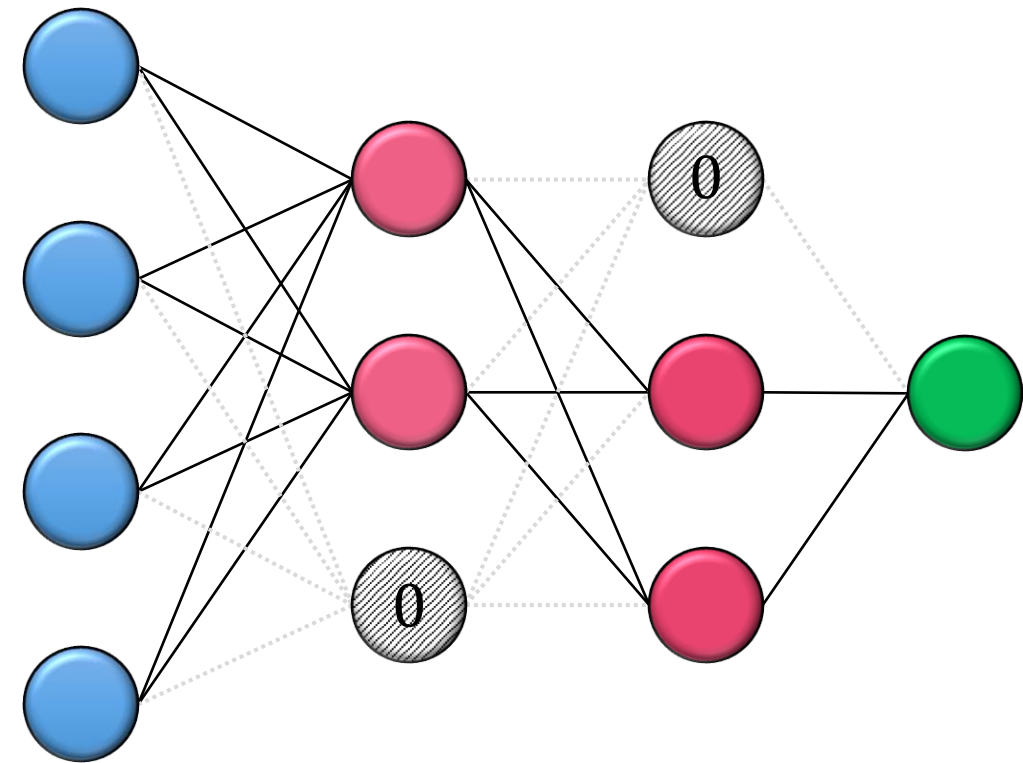
- ❑ L_2 also introduces a weight penalty in Loss calculation
- ❑ L_2 shrinks small (less important) weights **close to zero**
- ❑ L_2 penalizes strongly very large weights
- ❑ L_2 encourages the model to use all parameters more evenly

$$L_2 = \text{Loss function} + \underbrace{\lambda \sum_{i=1}^n w_i^2}_{\substack{\text{Penalty term} \\ \text{Adds a bias to} \\ \text{avoid overfitting}}}$$

- λ is a parameter that determines the strength of the regularization
- $\lambda = 0$: no regularisation
- λ can be tuned with hyperparameter tuning

3 Dropout

- ❑ **Dropout** is one of the most effective and most commonly used regularization techniques in deep learning
- ❑ Dropout randomly “drops” (sets to zero) a fraction of neurons during **each training iteration**
- ❑ Dropout prevents neurons from co-adapting too strongly. Forces the network to not rely on specific neurons and to generalize better

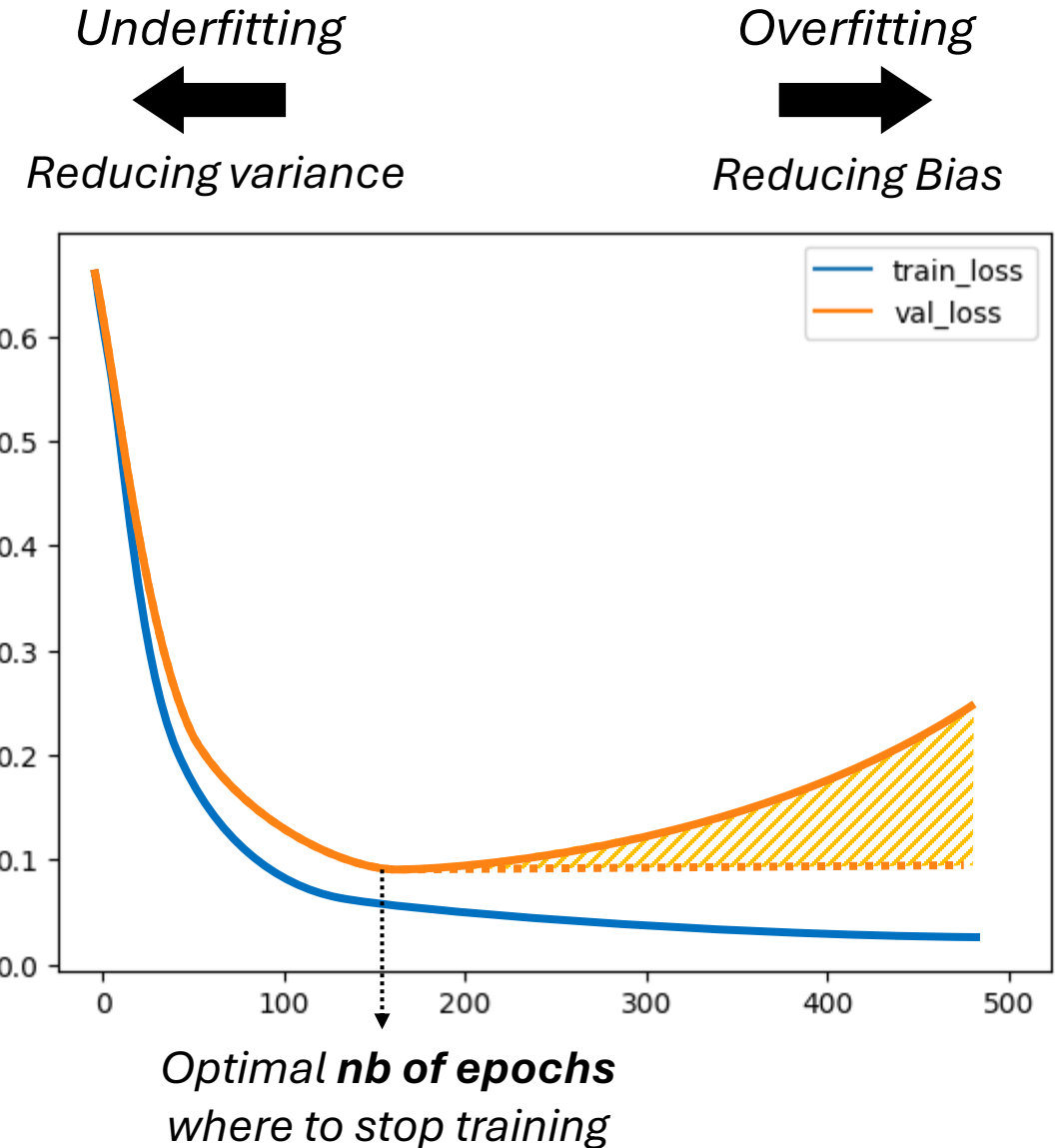


- p is a parameter that determines proportions of neurons to drop
- $p = 0$: no dropout
- p can be tuned with hyperparameter tuning

4

Early stopping

- ❑ **Early stopping** is a simple but extremely effective technique
- ❑ Principle is to monitor model performance on a validation set during training and to stop when **validation error stops decreasing**
- ❑ Early stopping stops training before the model memorizes training data and overfits

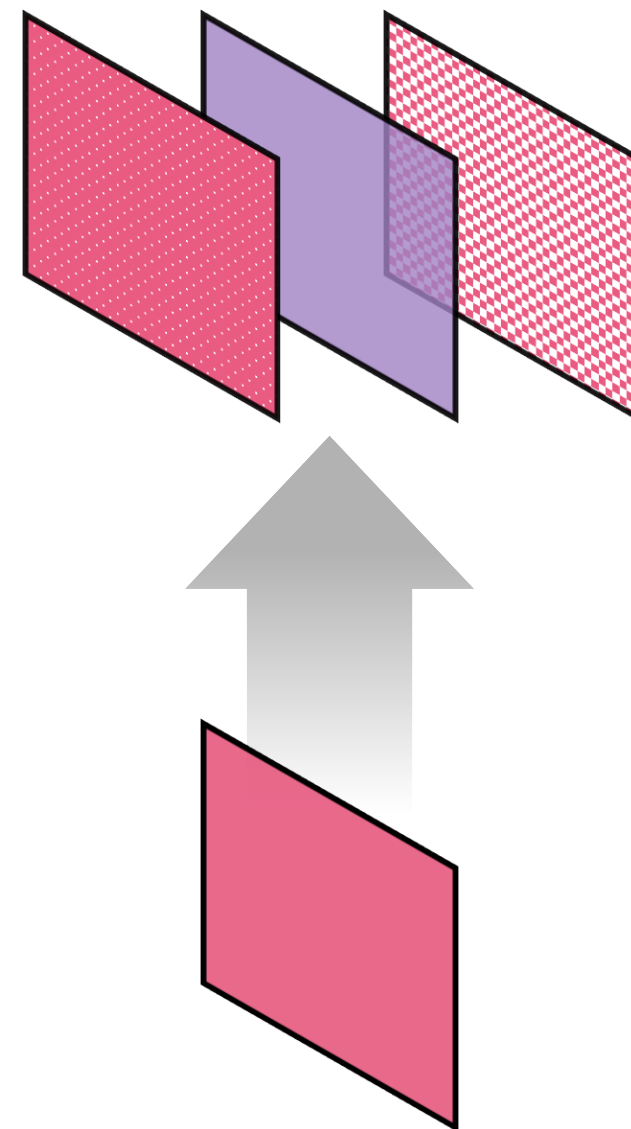


5

Data Augmentation

- ❑ Collecting more data to train a model is always a good deal. It reduces variance without increasing bias
- ❑ But available data is not infinite

- ❑ Collecting more data to train a model is always a good deal. It reduces variance without increasing bias
- ❑ But available data is not infinite
- ❑ **Data Augmentation** simulates new data by creating modified versions of training data with a number of random transformations, without changing labels
- ❑ The model learns to be invariant to irrelevant variations in the input and then generalizes better
- ❑ Data Augmentation is massively used in Computer Vision and Natural Language processing fields



Base Augmentations

Geometry based



rotate



shear



vertical-flip



horizontal-flip



crop



crop-and-pad

Perspective-
transformElastic-
transformation

Color based



sharpen

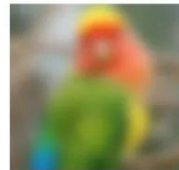


brighten

Gamma-
contrast

invert

Noise / occlusion



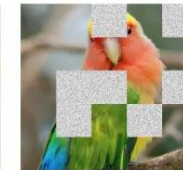
gaussian-blur

additive-gaussian-
noise

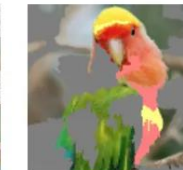
translate-x



translate-y



coarse-salt



super-pixel



emboss

Weather



clouds



fog



snow-flakes

Fast-snowy-
landscape

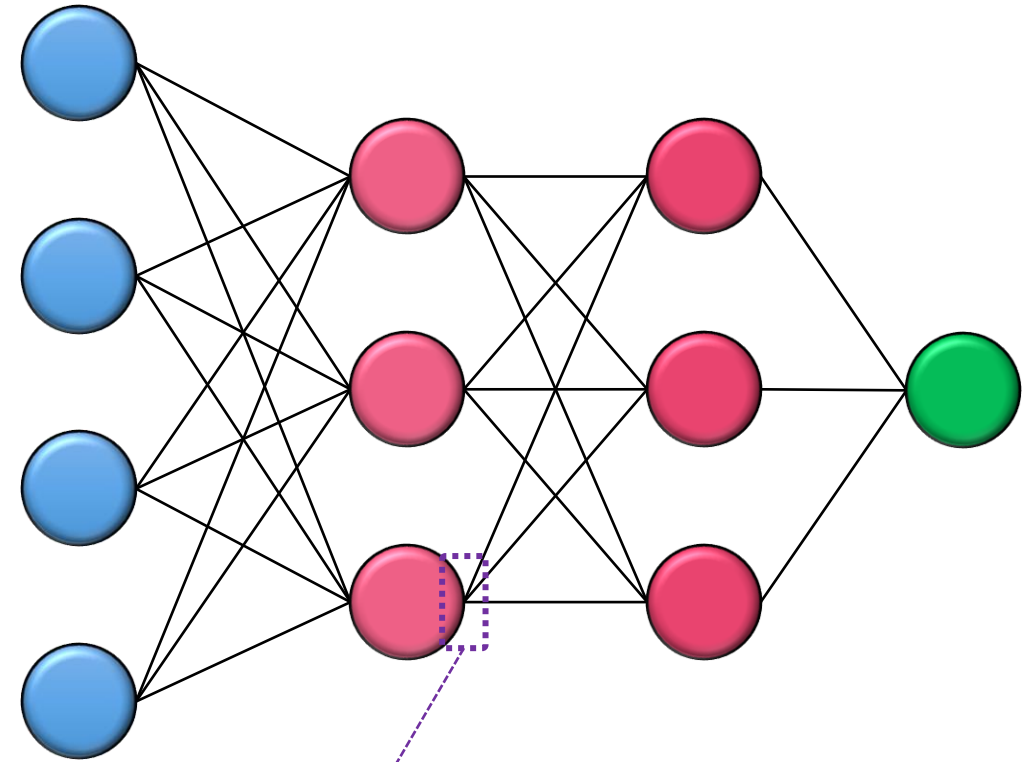
5

Data Augmentation

Random Deletion	Random Insertion	Random Replacement
“The scientist [...] the experiment carefully”	“The scientist zenith conducted the experiment carefully”	“The scientist zenith the experiment carefully”

Random Swap	Back Translation	Generation by LLM
“ conducted scientist The the experiment carefully”	“ The scientist conducted the experiment meticulously ”	“ The scientist performed the experiment with great caution ”

- ❑ **Batch Normalisation** is a technique that stabilises the distribution of inputs to each layer during training
- ❑ Batch Norm normalises each neuron's activations across the mini-batch
- ❑ It keeps activations centred and scaled during training, stabilizing gradients and allowing higher learning rates



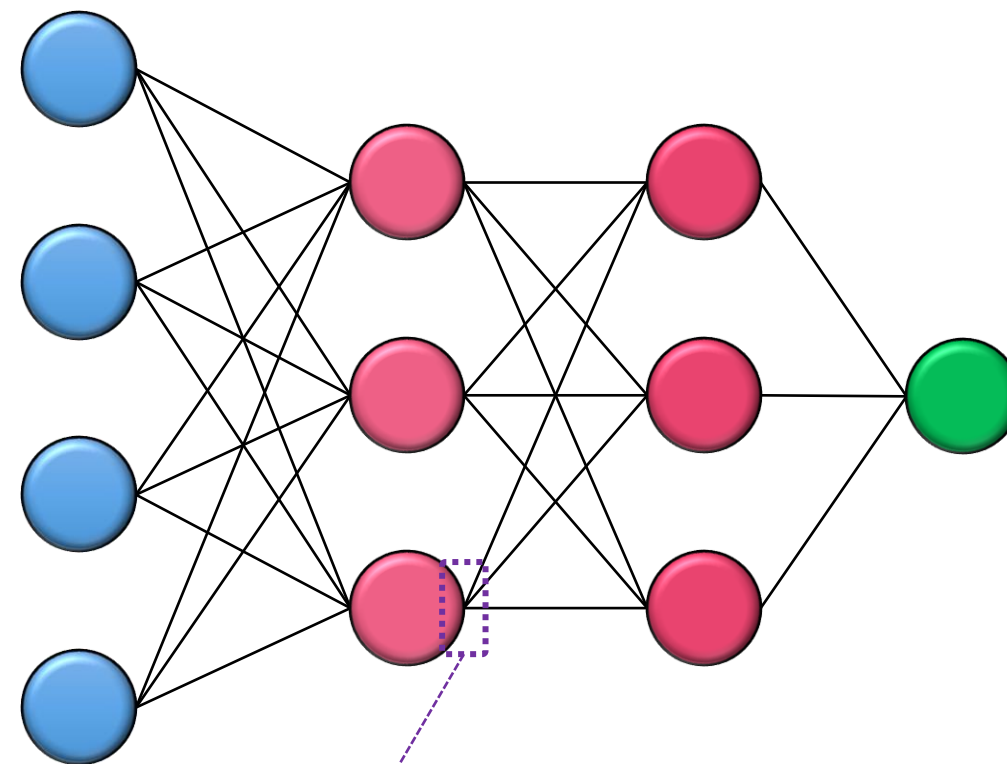
*Z-Normalization of
mini-batch
activation outputs*

$$x' = \frac{x - \mu}{\sigma}$$

6

Batch Normalisation

- ❑ Prevents the problem of vanishing gradient
- ❑ Can effectively control a poor weight initialization
- ❑ Significantly reduces the time required for network convergence
- ❑ Fights a Deep Learning problem called “**Internal Covariate Shift**”



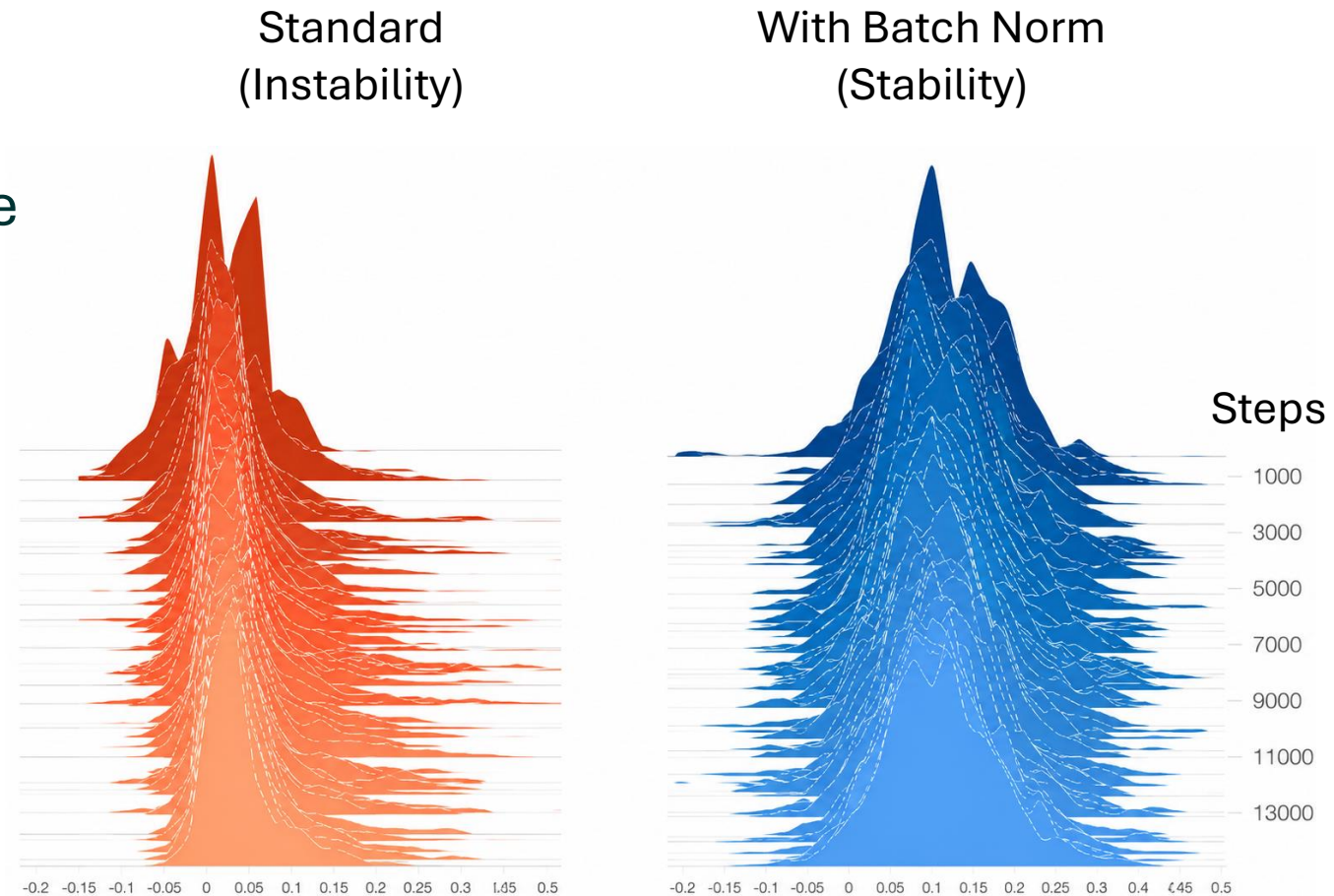
*Z-Normalization of
mini-batch
activation outputs*

$$x' = \frac{x - \mu}{\sigma}$$

❑ Internal Covariate Shift

- Is the change in the distribution of inputs inside the network due to the change in network parameters during training
- The data received by each **deep layer** is constantly shifting, making it harder for the layer to learn because they have to continuously adapt to new input distributions
- The problem that gets bigger as a Neural Networks goes deeper

❑ Batch Normalization stabilizes next layers' inputs



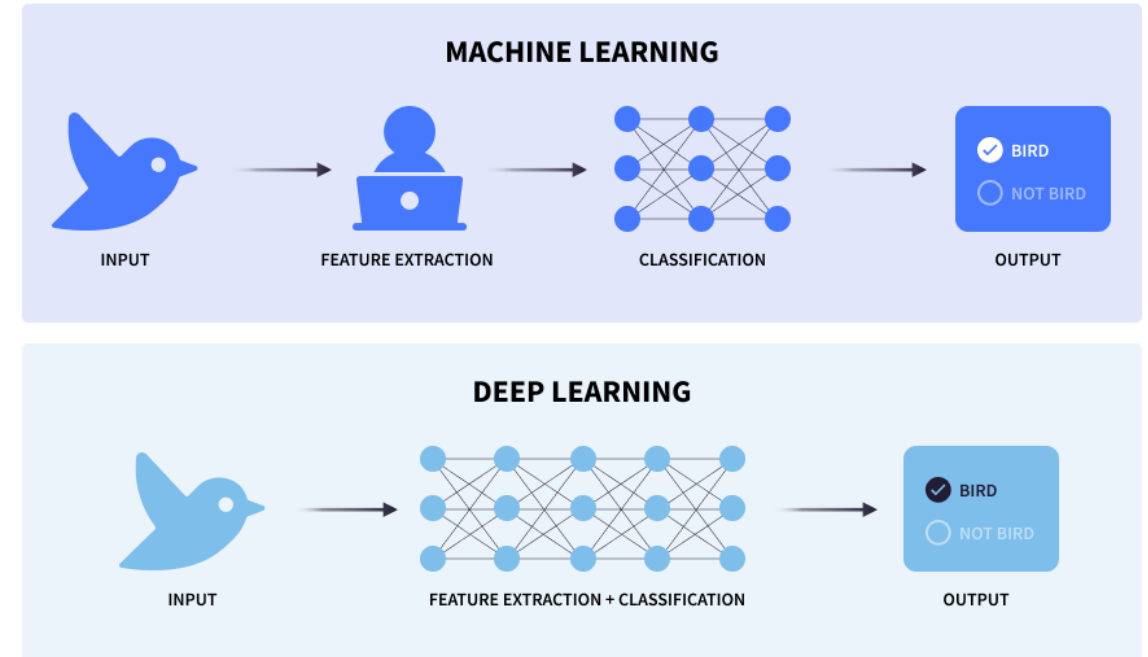
PDFs of activations over layer #11 (VGG Net)
(= next layer's inputs)

Santurkar et al. (2018) - How does batch normalization help optimization?

Why choosing Deep Learning?

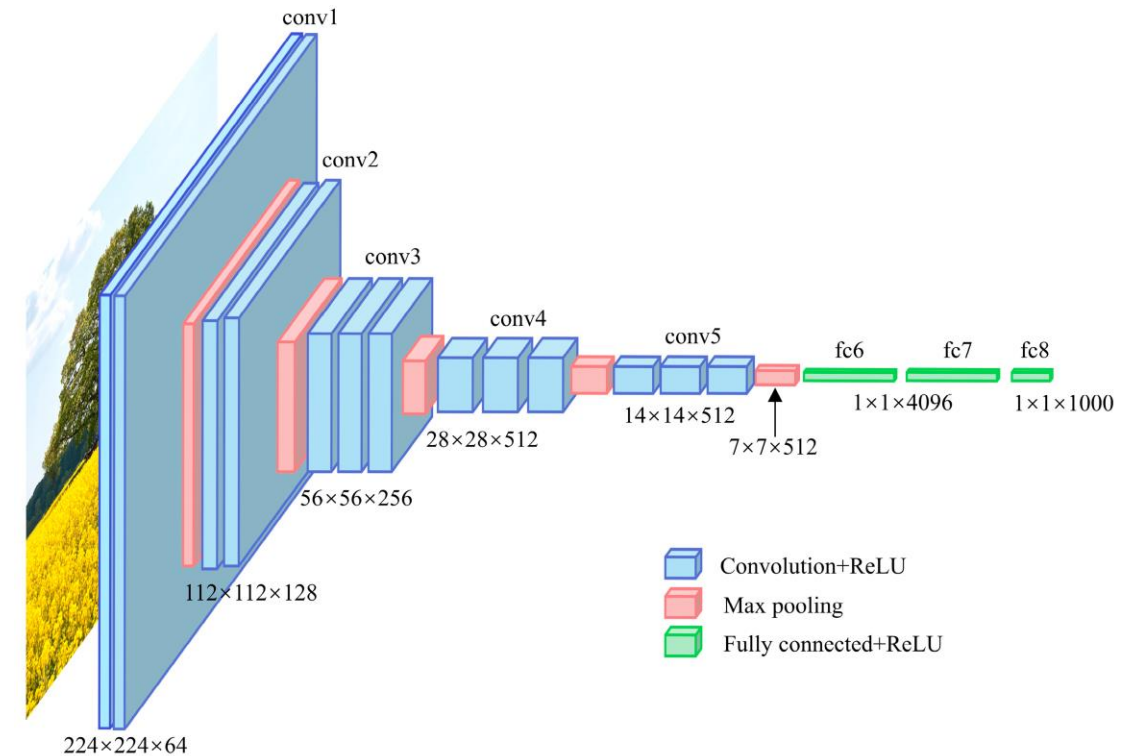
Why choosing Deep Learning?

- ❑ Deep learning models can automatically learn to extract features directly from raw data, reducing the need for manual feature engineering
- ❑ Features are inside the network!
- ❑ This make them very powerful for applications involving unstructured data (images, audio, text...)
- ❑ Because of their capacity to learn complex non-linear relationship between features, they are chosen over classic ML methods **to solve most complex problems**



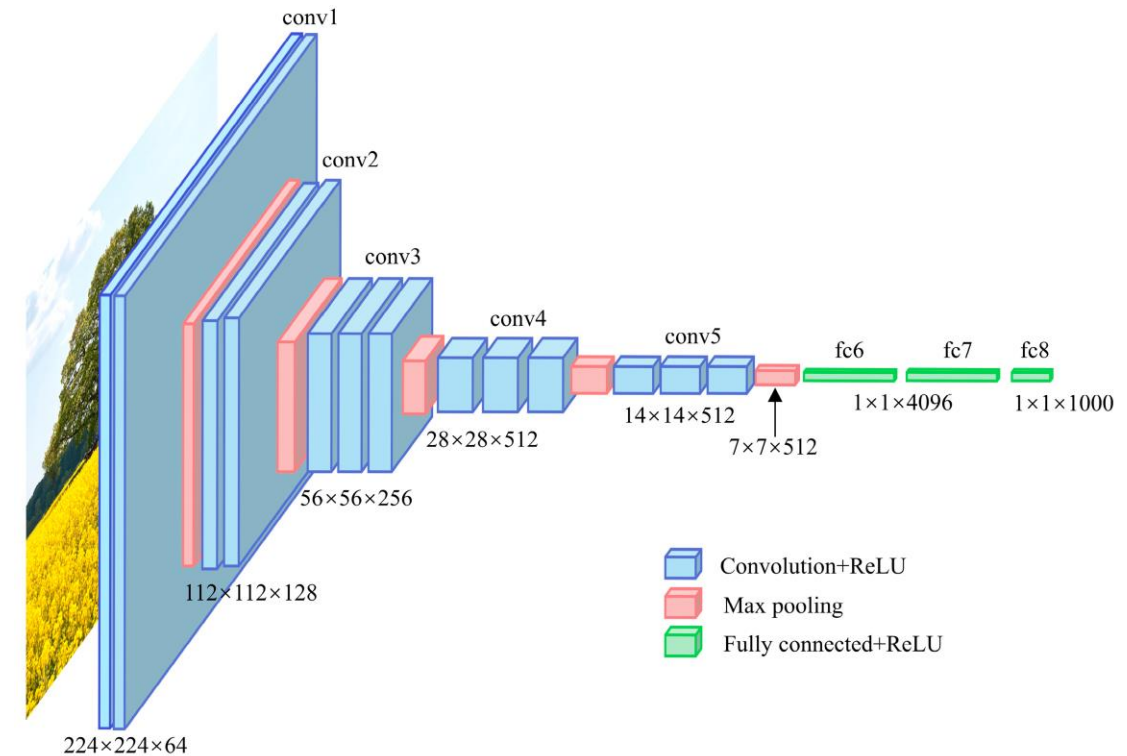
Why going deeper?

- ❑ Each layer learns progressively more abstract representations
 - Lower layer learns simple patterns
 - Higher layers learn complex concepts
- ❑ Stacking layers allows the network to compose features hierarchically, which is more efficient than a single wide layer



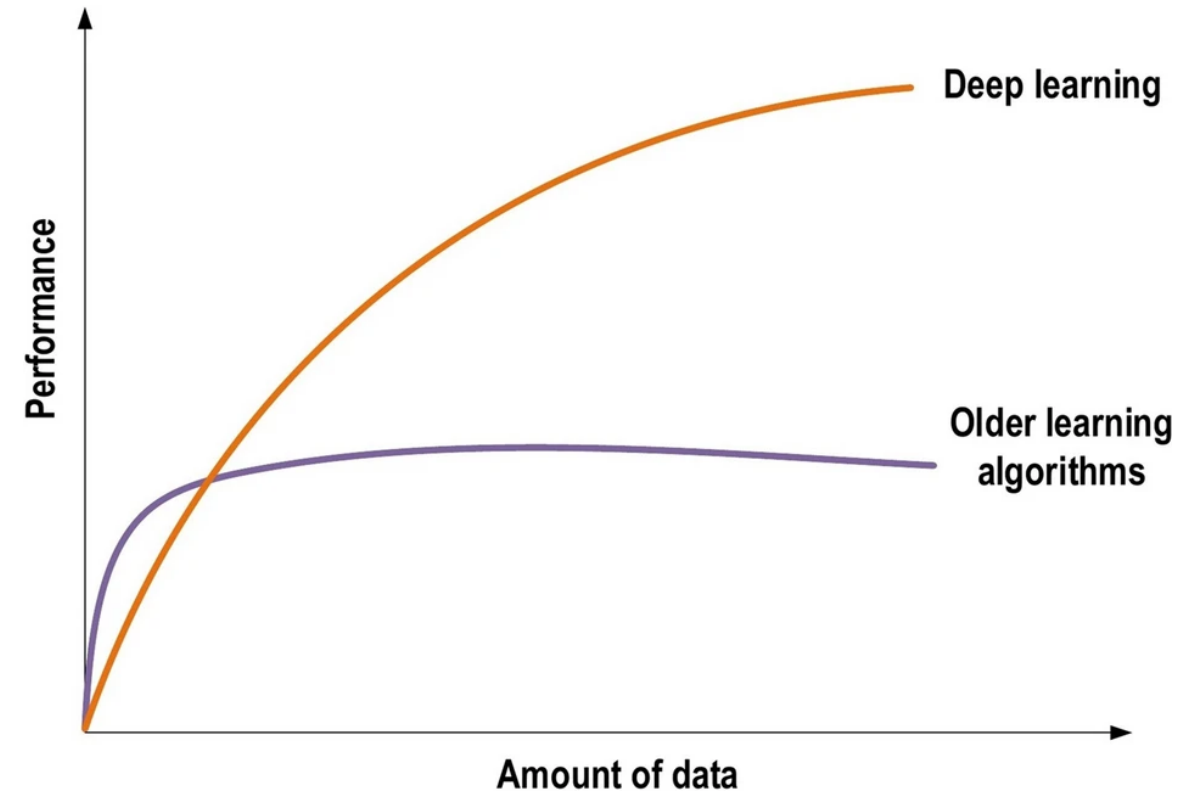
Why going deeper?

- ❑ Each layer learns progressively more abstract representations
 - Lower layer learns simple patterns
 - Higher layers learn complex concepts
- ❑ Stacking layers allows the network to compose features hierarchically, which is more efficient than a single wide layer
- ❑ Deep architectures can reach better performances using fewer total parameters than a single huge hidden layer
- ❑ This hierarchical feature extraction is the main reason deep networks outperform shallow ones on complex tasks



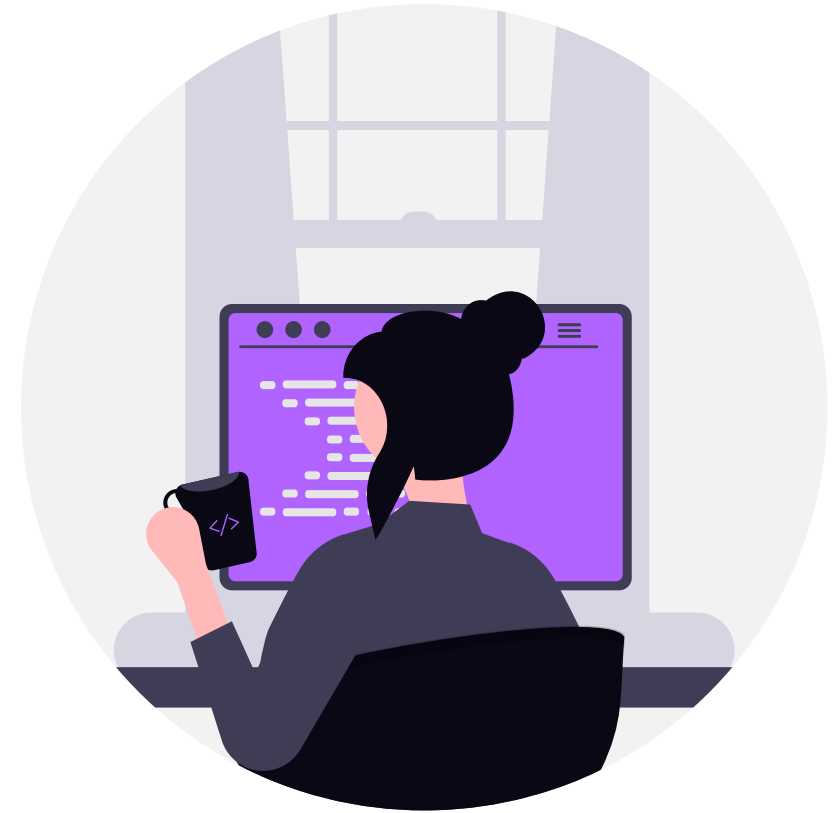
Deep Learning drawbacks

- ❑ They require huge amounts of accurately labelled data **because they also learn their own feature representations**
- ❑ They are prone to overfitting when data is limited or not diverse enough
- ❑ They require a lot of computational power (GPU) and take time to train
- ❑ As they are modeling very complex situations with non-linearity and interactions between inputs, they are nearly impossible to interpret (black boxes)



Training deep networks is challenging

- ❑ **Vanishing / exploding gradients / Dying ReLU** make it difficult to propagate learning signals through many layers
- ❑ **Internal covariate shift** slows training
- ❑ **Overfitting** risk increases with network capacity
- ❑ Optimization landscapes become complex (saddle points, local minima)
- ❑ Requires careful initialization, normalization, and specialized optimizers
- ❑ Data Science's role is to find the best compromise!



That's it!

***Hope you enjoyed this
course***

Thank you all